OTTO VON GUERICKE UNIVERSITÄT MAGDEBURG
INF FACULTY OF COMPUTER SCIENCE

Department of Simulation and Graphics

Master Thesis

in partial fulfillment of the requirements for the degree of

Master of Science (M.Sc.)

Automatic Landmark Detection for Preoperative Planning of Hip Surgery using Image Processing and Convolutional Neural Networks

Author	Benjamin Bergner
Student number	195598
Submitted on	February 22, 2018
Primary Reviewer	Jun.-Prof. Dr. Christian Hansen
Secondary Reviewer	Adj. Assoc. Prof. Dr. Claes Lundström

Abstract

Preoperative planning software helps orthopedists being well prepared for hip surgeries. Implants can be virtually positioned specific to the patient's anatomy prior to the actual operation.

Currently, landmarks (points on a x-ray) must be placed manually by the surgeon in a time-consuming and error-prone fashion to derive implant locations. Automating landmark detection helps to automate the whole planning process in a fast and accurate manner. 2 different image processing and 2 convolutional neural network (CNN) approaches are therefore described and evaluated to detect 6 landmarks automatically on a clinical dataset of 1054 pelvis x-ray's.

A cascaded regression CNN (CCNNR) achieves a mean radial error of 2.34 ± 2.5 mm and a leg length discrepancy error of 2.2 ± 2.55 mm in an average time of 2.81s per image (unoptimized, CPU). The CCNNR can hereby be applied to different 2D planning tasks. On another skull x-ray dataset (IEEE ISBI Challenge 2015 for automated cephalometry [1]), it outperforms previous approaches in all but one measure. Furthermore, a heuristic for cascade definition is presented to detect landmarks even in high-resolution images and with small dataset sizes by continuously and smoothly simplifying the task.

Statutory Declaration

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

This thesis was not used in the same or in a similar version to achieve an academic grading or is being published elsewhere.

Magdeburg, February 22, 2018

(Signature)

Acknowledgment

I foremost want to thank Mattias Bergbom for giving me the possibility to intensify my machine learning studies in the digital health domain and for managing everything necessary in order to conduct this thesis successfully.

I want to thank my advisors Christian Hansen and Claes Lundström as well as Olof Sandberg for giving me valuable feedback on thesis structure and methodology. Furthermore, I want to thank Daniel Forsberg for helpful discussions on machine learning.

Without Sectra's orthopedic department, it would have been difficult (and less insightful) to annotate over 1000 x-rays, thank you.

Last but not least, I want to express my gratitude to everybody who encouraged and welcomed me warmly during Sweden's cold winter season.

Contents

1 Introduction

Total hip replacement (THR) is a common orthopedic surgery. In 2015, most people were operated in Switzerland, Germany and Austria (308, 299, 271 per 100.000 population) [2]. Since 2000, the rate has increased by 30% on average (OECD29). As one of the ten most disabling diseases in developed countries, osteoarthritis is often the reason for this surgery and is mostly performed on people above 60. Osteoarthritis is the degeneration of cartilage in the joint space between caput and acetabulum. It results in pain due to bone rubbing (compare both pelvis sides in figure 1.1 left). In THR, cartilage and damaged bone are removed. Femoral head and stem are replaced by prostheses. Diseased cartilage is reamed out and exchanged with an acetabular prosthesis called acetabular component or cup (see figure 1.1 right).[1]

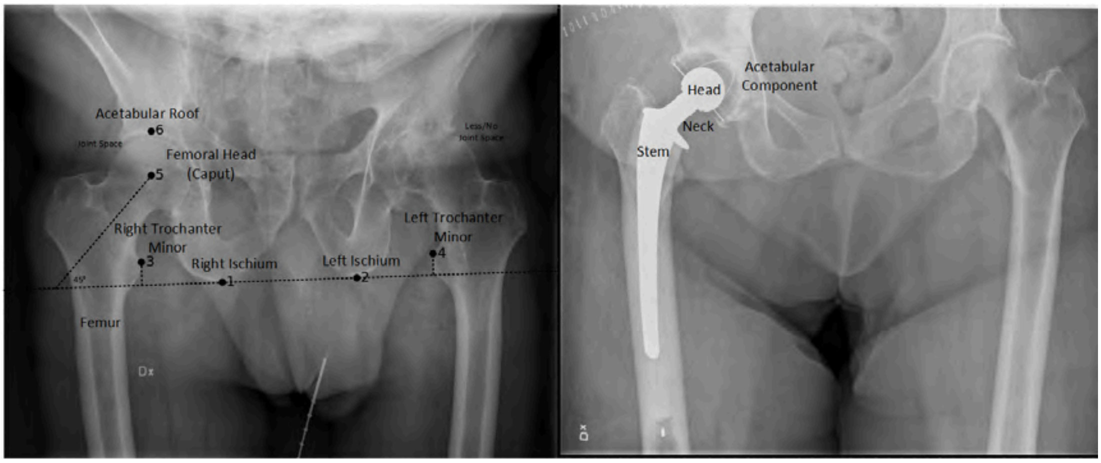

Figure 1.1: Two different pelvis x-rays; left: Landmarks and corresponding anatomy, right: Surgical outcome showing prostheses

Before entering the operating room, surgeons plan on the patient's x-ray to fit implants to the individual anatomy. It includes the selection of prostheses types

[1]More information about THR, see [3]

and sizes as well as their location and orientation relative to the anatomy. Leg length discrepancy (LLD) shall be compensated.[2] Plans can subsequently be viewed on a monitor during operation and overall ensure that surgeons are well prepared.

THR planning on x-ray images is one of the features of Sectra's preoperative planning suite, OrthoStation. The process can be divided in the following steps:

- X-ray import
- Automatic marker detection for correct distance scaling within the image
- Manual and guided landmark placement
- Automatic calculation and setup of prostheses sizes and locations
- Manual correction and adjustment of components

With automatisms, planning is facilitated and surgical outcomes improve. One of the last repetitive manual steps consists in placing landmarks on a x-ray. Landmarks are salient and defined points used by humans and machines for application-specific orientation. In orthopedics, they are needed to give automatic suggestions for prosthesis positioning, alignment and standardized sizes.

Six important anatomical landmarks for THR planning are both sides of the ischium and trochanter minor as well as the caput center and acetabular roof (see figure 1.1 left). The first two landmarks are defined as lowest ischium points that form a tangent to it and accumulate equal area under the convex anatomical curves on both sides. The landmarks on the trochanters minor are defined as upmost points heading to the image center. In some cases, these definitions can apply to various locations (see also chapter 4). The caput ideally has the shape of a circle within a 2D image that we want to capture. Landmark 5 is the caput center whereas landmark 6 lies on the vertical shift to it and touches the acetabular roof.

With the first 4 landmarks, LLD can be calculated. This measure serves for automatic adjustment of prostheses sizes, positions and their relations to each other to compensate for unequal leg lengths. It is the difference between the line

[2] For more information about the value of preoperative planning in THR surgery, see [4]

lengths from the trochanters minor perpendicular to the line spanned by right and left ischium (see figure 1.1 left). The distance between caput center and acetabular roof can be considered as cup radius and hence determines its size. In addition, landmark 5 specifies orientation of cup (and as a consequence thereof the stem) as 45° between ischium line and caput center. Since the caput is not always similar to a circle (see appendix A) and cup positioning is subjective to the surgeon's choice, finding the last two landmarks is regarded as an extra for this thesis and can only give default cup placements that might need adjustments as personally desired.

We can conclude that landmarks play a crucial role in THR planning. Detecting them automatically (called automatic landmark detection or ALD) provides surgeons (and patients) and Sectra various advantages as listed in table 1.1.

Surgeons	**Business development**
Decreases planning time	Happier customers
Increases planning quality in stressful clinical daily routine	Follows automation chain in orthopedic solution
Increases planning quality for inexperienced surgeons	Application scalable (hip, knee, shoulder, spine)
Less attention overhead	Expandable to 3D planning (manual placement difficult)
Fewer clicks leads to better UX	Usable as basis for segmentation (common image analysis task)
Wow effect	Usable for marketing (customers, potential customers, future employees)

Table 1.1: Advantages of automatic landmark detection for orthopedic surgeons and business development

Manual placement is error-prone due to image quality, landmark definition ambiguities and dependent on the surgeon's experience in his profession as well as with the software. In addition, it increases planning time and attention overhead by following a guide. Even though manual setting can be accurate, time restrictions in clinical daily routine reduces time spent on this mundane task and hence leads to more errors. From a business development point of view, ALD is valuable as it has wide applicability within the orthopedic suite and can be expanded to 3D planning.

The goal of this thesis is to investigate the possibilities to detect landmarks automatically on hip x-rays. Those can then e.g. be suggested to the surgeon or utilized to fully automate planning. Therefore, simple image processing as well as machine learning techniques are tested. The research will be conducted in 2D to reduce complexity in a new field for the company. In addition, the thesis outcome is aimed to be transferable to production and hence must be reasonably fast.

In chapter 2, related work will be reviewed. In a bigger chapter 3, all tested methods are explained. Chapter 4 gives more information about the data used in order to train and test models while chapter 5 focuses on implementation details. In chapter 6, all methods are evaluated and chapter 7 discusses these results by comparing all approaches and gives insights for production usage. The outcomes are subsequently concluded and future work recommendations are presented in chapter 8.

2 Related Work

ALD is a common procedure in medical imaging. It is mostly used as a preliminary step for segmentation, as a means for registration algorithms and to measure geometric relationships for anatomical analysis. The latter is the application in this thesis. Especially for hips, only few publications exist.

In [5] e.g., the authors detect landmarks to support atlas-based segmentation of pelvis and femur in CT images within a computer aided diagnosis and planning system for periacetabular osteotomy. As predictor, random forest regression is used. Around every landmark in the training data, sub-volumes are sampled. Subsequently, mean intensity and variance of volume blocks as well as displacements from block centers to the target landmark are calculated. Every tree then votes for a landmark location for random sub-volumes in test images. Finally, a Gaussian probability map is constructed to determine the best fit position. Similar to the previous approach, [6] apply random forests with Haar-like features to first detect objects in a global search followed by a local search. Their goal is to find femur boundary points for segmentation. In [7, 8], the authors apply atlases to segment bone structures and detect landmarks for 3D hip operation planning. An expired Siemens patent [9] describes their invention of automatically finding landmarks for THR surgery by intensity ridge maps within a computed region of interest. Previous work for the hip is either used for segmentation, in 3D or implements vague image analysis methods. In this thesis, new approaches for 2D hip landmark detection instead of 3D are described and evaluated on a clinical dataset especially for integration in planning software for hip surgery. Besides conventional machine learning methods, two convolutional neural network variants are evaluated.

Apart from anatomical landmark detection in hip images, several applications for other parts of the body have been investigated. [10] e.g. use a template matching approach to find landmarks in chest images as initial registration of two CT scans. [11] apply landmark detection to MR brain image registration. Besides using random forests, they propose point jumping to iteratively refine vote positions. Another example is shown in [12] where aortic valve landmarks are identified for planning and visual guidance during surgery.

One recent publication [13] implements a CNN to find landmarks for radiographic assessment of adolescent idiopathic scoliosis. They implement boost layers for feature outliers detection and alignment as well as anatomy shaped regression outputs. Another interesting application due to its similarity is anatomical landmark detection on skull x-rays for automated cephalometry. Two public challenges were organized and results are reported in [1, 14]. An open small high-resolution 2D dataset has been provided and several authors evaluate their models on this data. One recent paper [15] makes use of classification CNNs and a shape model [16] to find landmarks even with few training images and achieves best performance among other competitors within 2 mm distance bound. One of the methods in this thesis will be evaluated on this dataset and applies a CNN regression cascade suitable for small datasets without the dependence on an explicit shape model. In addition, it provides fast predictions.

Further deep learning approaches on medical data in 3D are reviewed in [17]. Besides use-cases in the field of medicine, another important ALD research branch resides in images depicting faces (e.g. mouth corners, nose tip, eyes) and humans. Some applications in this field are face detection, pose estimation [18], facial expression analysis [19], augmented reality [20, 21] and modeling of age progression [22]. Even though not industry-related, the task of finding landmarks far away from each other and dimensions (2D) are comparable to hip images in this thesis (in contrast to most medical ALD research). Deep learning approaches are common in this domain. One of the reasons is the bigger amount of available data. In [23], a cascade of regression CNNs is used for coarse-to-fine prediction of 5 facial landmarks in a training set of 10.000 images of size 39x39 px. Another typical strategy is multi-task learning [24, 25, 26]. Multi-task learning assumes that features learned for a related task are useful in predicting outputs for the main task. In [24] e.g., comparable results to the cascaded approach are achieved with CNNs learning head pose, gender, age, facial expression and attributes as well as facial landmarks at the same time. Other complex methods in this industry encompass deep deformation networks [27] and adversarial learning [28].

An abundance of training data even for small images is typical in these papers because finding landmarks in faces can be difficult due to variances like people wearing glasses, occlusions caused by hair, different head poses and expressions. The clinical hip dataset is much smaller and comprises high resolution images but methods do not need to handle above mentioned hurdles to the same extent. In this thesis, it is shown that the idea of using cascades is also suitable on very distinct dataset properties including visual features, variance types, dataset and image sizes. Furthermore, a heuristic is presented to estimate cascade hyperparameters (size of refinement regions, restricted augmentation shifting) that make learning on small datasets and big images possible.

Besides the previously mentioned contributions, the thesis provides a suitable approach for production usage, a beginning path in machine learning for Sectra's orthopedic department and educational insights about the complexity of such problems, how and how not to solve them.

3 Methods

A landmark inside a x-ray image is represented by a single pixel. The number of possible solutions corresponds hence to the amount of pixels (potentially millions). It would facilitate the detection process to shrink the search space into a region of interest (ROI). For ROI computation and within it, visual features are extracted from a set of images $\mathcal{I}_{train}$ to describe landmark locations $l \in \mathcal{L}_{train}$ and their surrounding. The combination of $\mathcal{I}_{train}$ and $\mathcal{L}_{train}$ is called the training dataset $\mathcal{D}_{train}$. With the help of those visual features, landmark positions $\hat{l} \in \hat{\mathcal{L}}$ can be estimated in images $\mathcal{I}_{test}$ sampled from a distinct set, called the test set $\mathcal{D}_{test}$. A third validation or development set $\mathcal{D}_{val}$, might be included to tune hyperparameters.

Local visual features might occur similarly in different areas within an image (imagine the pelvis symmetry). Therefore, the extraction of multiple estimates increases the chance of encountering the true location. Those estimates are called candidates $\mathcal{C}_l$ of a landmark l. Furthermore, anatomical landmarks show similar geometric relationships (distances, angles) between each other, e.g. the right trochanter minor is (hopefully) always located right to the left one. We can exploit these relationships with shape models to narrow down candidates to a final estimate. The landmark detection process can consequently be divided in three steps:

1. ROI extraction
2. Landmark candidate selection based on visual features
3. Final estimation by shape modeling

In order to solve the landmark detection problem, descriptions of four strategies follow that make use of these steps in different ways. The first two, contour change points and recursive regions of interest, are simple approaches based on intuition and mere image processing that utilize hand-crafted features. The latter two, CNN classification and cascaded CNN regression, are deep learning models that learn features by themselves and make decisions based on them. While the first three methods apply a shape model with explicit statistical analysis, the last relies on an implicit model.

3.1 Contour Change Points

Due to high ambiguity of cup placements and high variances in the caput region, the main task has been restricted to find the first 4 landmarks. Imagining the pelvis contour, the trochanter minor and ischium landmarks lie on steep contour changes. The question of landmark detection can therefore be shifted to contour change point (CCP) detection. Figure 3.1 gives an overview of the approach which divides into image preprocessing, feature extraction, feature storage and prediction.

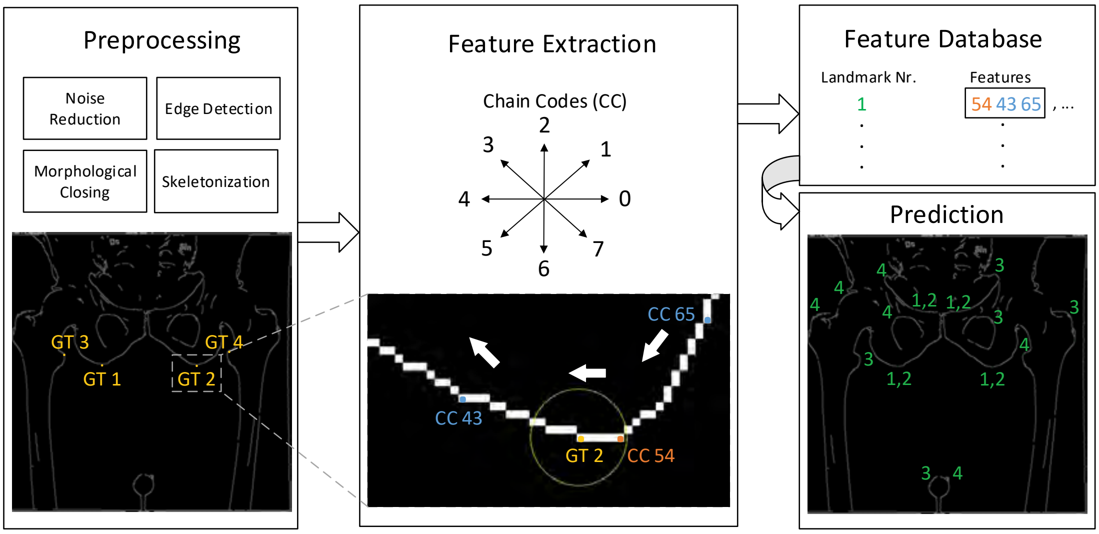

Figure 3.1: Contour change point approach overview

As first preprocessing step, histogram equalization is applied to all images ensuring clearer contrast, i.e. a higher intensity gradient, around the landmark area. Gaussian smoothing follows to reduce noise. Then, edges are detected with the Canny algorithm [29]. Two thresholds t_{min} and t_{max} determine which parts of the image will be considered as edges. Sobel kernel based gradient magnitudes below t_{min} are non-edges, over t_{max} true edges and gradient magnitudes between the thresholds are considered as edges if they connect to true ones. Thresholds underlie dynamic adjustments (see 3.1, 3.2) computed with the median image intensity m_i in the range $[0, 1]$ and d_{min}, d_{max} as hyperparameters.

$$t_{min} = max(0, (1 - d_{min}) \cdot m_i) \tag{3.1}$$

$$t_{max} = min(1, (1 + d_{max}) \cdot m_i) \tag{3.2}$$

In order to connect nearby edges, morphological closing (dilation followed by erosion) is applied. Since this operation introduces several filled areas, skeletonization subsequently decreases the contour thickness to a single pixel again. The effects of each preprocessing step is depicted in figure 3.2.

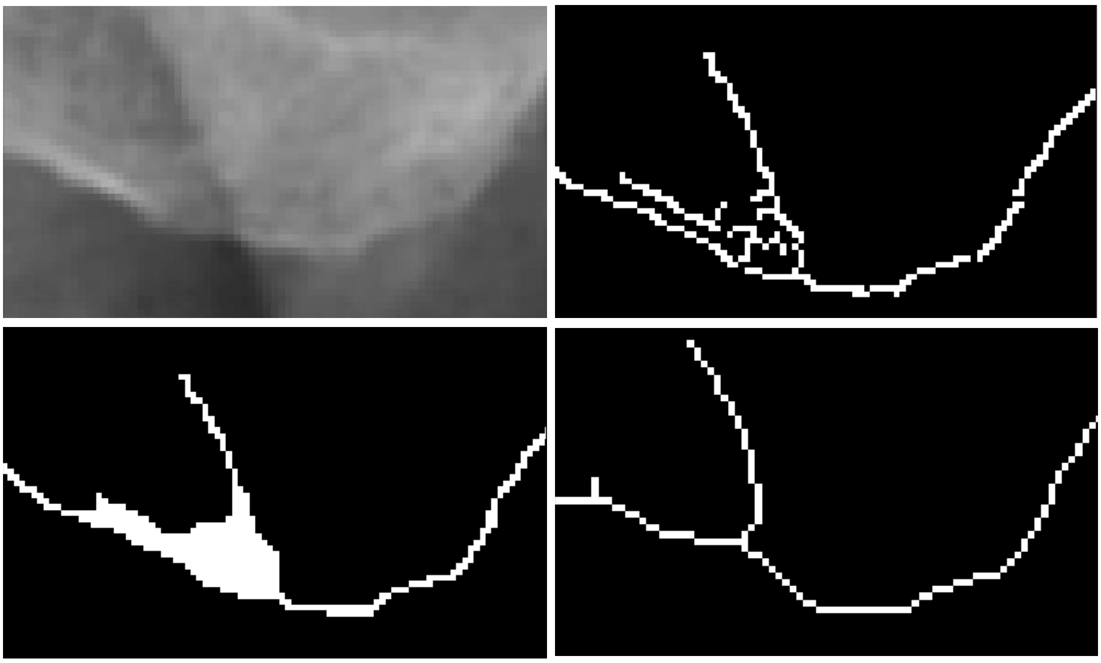

Figure 3.2: Preprocessing steps exemplified on left ischium, top left: Gaussian smoothing blurs histogram equalized image and hence reduces noise, top right: canny edge detection, bottom left: morphological closing, bottom right: skeletonization

After contour lines have been made visible, they need to be converted into a machine-readable format for feature extraction. Chain codes (CCs) [30] for horizontal, vertical and diagonal directions can be used to translate contours into numerical strings. A contour finding algorithm returns the coordinates of all direction changes for each background separated contour as a prerequisite for the subsequent pixel-wise transcription.

Several considerations must be addressed for coordinate to numerical string transformation:

- Starting coordinates of each contour must be determined. Pixels lying at the coordinate extrema with only one neighbor are favored.
- A contour pixel can have multiple 8-connected neighbors, e.g. when lines cross (see figure 3.2 bottom right). Due to line crossings, a contour can consist of several chains.
- Already passed coordinates must be blacklisted to circumvent infinite loops in circular contours.
- A traversing orientation (e.g. clock-wise) must be set.

Given extracted numerical chains (e.g. "00664422" for a rectangle), we are able to find change points (CPs). Each CP consists of two numbers, the first representing original, the second upcoming direction. The combination "54" e.g. symbolizes a direction change from down-left to left. As can be seen in figure 3.1, a CP is not placed at every direction change, rather only at locations of significant change to account for pelvis shape variance.

Each contour is traversed from a determined start point and the number of CC occurrences is counted. Once a CC hits a change threshold t_c, it will be set as first element of a CP. If t_c is not reached, the contour is too short and discarded. In order to find the second element, we continue traversing the contour and again count contributions of each CC. Codes far away from the first element in the code compass (see figure 3.1) are considered as contributing more to a change than nearby codes. Given a first element of e.g. CC 6, CC 0 would contribute twice as much to the change as CC 7. If the same direction as the first element is encountered, the magnitude of change is decreased by 1. If the change magnitude becomes negative, no significant change has been detected and all contributions are reset. However, if t_c is hit, the second element will be set as the CC with highest contribution. If several CC contributed equally, the nearest CC to the first element is chosen because it occurred most often. The coordinate of the CP is the first occurrence of the second element in the current session. The final CPs are finally orientation normalized (e.g. CC 46 should be equal to CC 20).

Once all CPs have been extracted, they need to be composed to features. A feature in this context is a combination of CPs. Within neighborhood radius $r_{\mathcal{N}}$ centered at the ground truth (GT) landmark position, CPs and their k nearest neighbors compose a feature. They are stored landmark-wise in a feature database FDB filled from all $i \in \mathcal{I}_{train}$. Features are utilized for prediction in a lazy fashion. In an $i \in \mathcal{I}_{test}$, features are extracted again and compared to entries from FDB. Whenever a match for a landmark is found, the initially empty $\mathcal{C}_l$ is updated as union with the respective CP coordinate of the found feature.

Conclusively, several variations for feature extraction are feasible to account for image variances and to avoid that too many features are found in a test instance. Some of them are:

- Allow similar CPs (e.g. "45", "46") as valid matches when comparing features in FDB and $\mathcal{I}_{test}$
- Collect k nearest neighbors but accept features with $\#matches < k$
- Reduce FDB size by only including most occurring features

3.2 Recursive Regions of Interest

The previous method does not benefit from search space reduction. Another intuitive approach that makes extensive use of this are recursive regions of interest (RROI). For our application, a true ROI is a subimage that possesses a landmark. It preserves the most important part of an image including characteristics around a landmark. Most importantly, it discards unnecessary parts. If found accurately, they significantly shrink the possible options. ROIs can be applied before the actual landmark detection algorithm. In case of the previously described CCP algorithm e.g., it could reduce the number of candidates since local CP features might occur many times within an image but will less in a smaller area. The risk of using ROIs is that if not found accurately, i.e. the region does not possess the landmark, subsequent detection algorithms cannot identify the true location anymore.

Once a ROI is found for a particular landmark, we can again simplify detection by discovering another ROI within the previous one. This concept can be continued in a recursive manner, not only to decrease the search space but to reveal the landmark itself. In this extended case, ROIs are applied until only one pixel remains and hence act as a landmark detection method. An example is depicted in Figure 3.3 left.

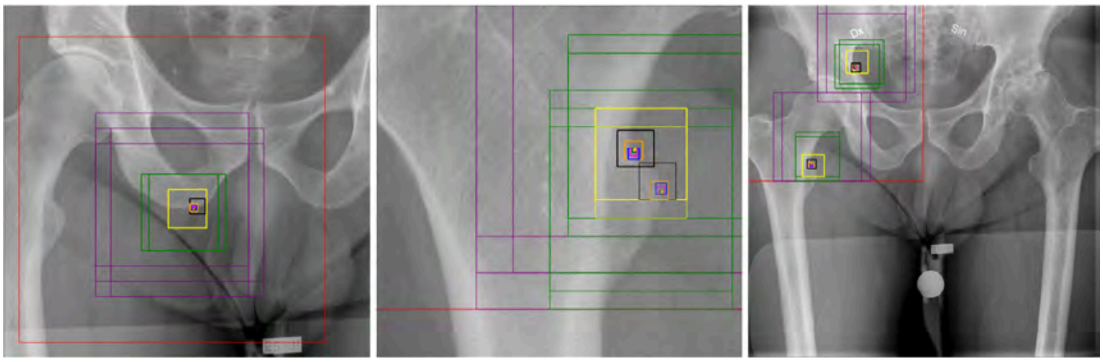

Figure 3.3: Landmark prediction with recursive ROIs; left: successful detection of landmark 1, middle: deep layers detect similar coordinates for landmark 3, right: shallow layers detect different locations for landmark 3

A ROI for a landmark l from a test image i_{test} is determined by comparing sampled patches $p \in P$ from i_{test} with a reference patch r_l of same size. This is called correspondence matching. One measure of comparison is the pixel-wise sum of squared differences (SSD) (see 3.3) with u, v as row and column iterators and s as region size. The ROI is then considered as sample with minimum SSD because it correlates best with the reference.[1]

$$SSD = \sum_{(u,v) \in s} (p(u,v) - r_l(u,v))^2 \tag{3.3}$$

[1] More about correspondence matching and similarity measures in [31]

Subimages as potential ROIs are sampled in a sliding window manner without considering samples that lie partly outside their source image. A reference patch (ground truth) of size s can be calculated as the pixel-wise mean μ_l^s over all training regions centered around the true landmark as in 3.4.

$$\mu_l^s(u,v) = \frac{\sum_{i \in \mathcal{I}_{train}} i(u,v)}{|\mathcal{I}_{train}|}, (u,v) \in s \tag{3.4}$$

The mean is a simple but appropriate choice because affine transformations like rotation and scaling are limited. Reasons for this are the similar acquisition with a x-ray machine and the restricted variance of human anatomy. In addition, we only look at patches centered around the true landmark coordinates, further limiting differences. Even though the pelvis anatomy can vary especially between humans in need of surgery, they still have the same basic shape. Figure 3.4 shows μ_l^s for $l \in \mathcal{L}$ over $\mathcal{I}_{train}$ for the biggest region size. The means make most common visual features visible and appear as blurred versions of original images.

A second step that simplifies recognition of similar patches is scaling intensity values of both reference and samples between $[0,1]$ (3.5). This treats every image equally by giving more importance to visual structures instead of regional brightness differences.

$$i_{norm} = \frac{i - i_{min}}{i_{max} - i_{min}} \tag{3.5}$$

There are several hyperparameters that control RROI:

- number of layers A
- The already mentioned layer-wise region size s_a,
- Layer-wise sliding window $stride_a$
- Spawns per layer $spawn_a$

The last layer must either have a size of one pixel or the center of the last region is postulated as predicted landmark. In deeper layers, s can be decreased as the chance to encounter similar structures decreases along with it. For every layer, not only new samples must be drawn respective to the previous region,

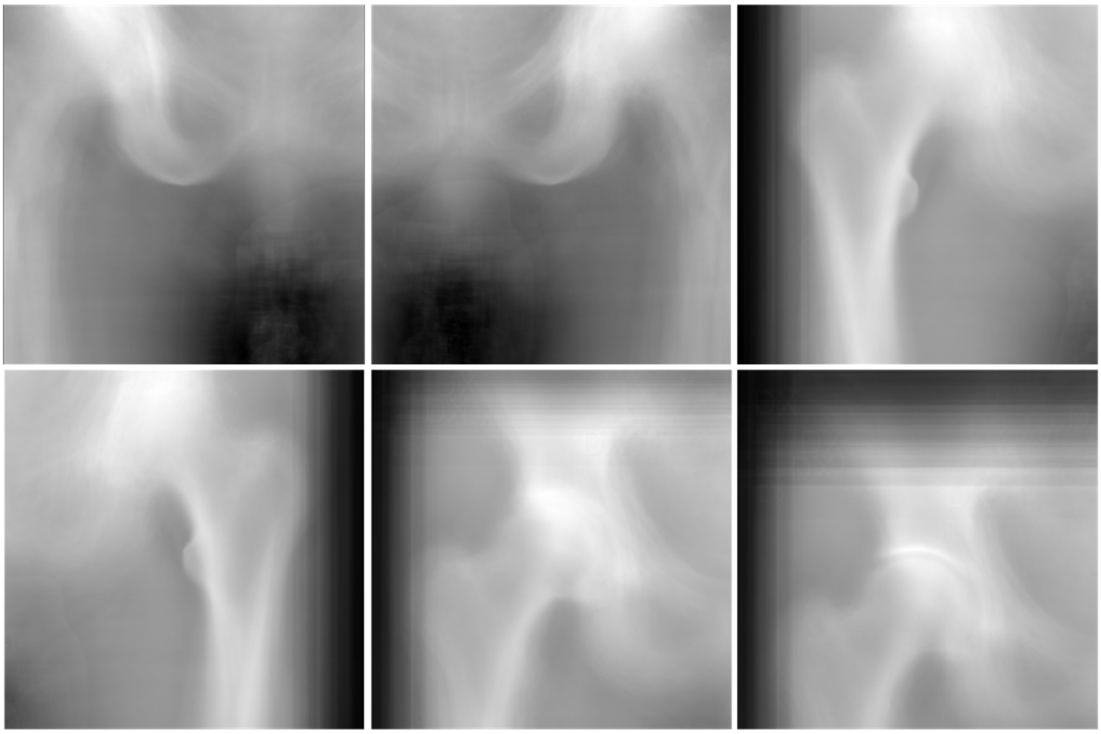

Figure 3.4: Mean image intensities over training set for landmarks 1 to 6

also intensity-adjusted means of equal size from $\mathcal{I}_{train}$ must be calculated (see Figure 3.5) for layer-wise correspondence matching.

s_a is assumed to be the number of vertical/horizontal pixels, defining a square, for layer a with $s_a > s_{a+1}$. The bigger s_a, the higher the probability that the true landmark will reside in the predicted ROI over images in $\mathcal{I}_{test}$. Is s_a too small, a patch similar to the true region might be predicted as ROI. The right ischium e.g. appears similar to the left ischium in a small region but becomes more dissimilar when symmetric structures like the trochanters minor become visible in a bigger region.

$stride_a$ controls how many pixels to move horizontally/vertically for sliding window patch sampling beginning at coordinate $(0, 0)$ in the top-left image corner. Patches that partly lie outside the bounds will be discarded. As layers become deeper and s_a decreases, $stride_a$ will decrease with it. The hyperparameter has two functions. First, the higher it is, the less computations need to be done. Setting it to 1 results in an exhaustive search. Second, $stride_a$ set reasonably high ensures that distant regions will be spawned, a wanted effect to reduce redundant candidates and to create very distinct hypotheses that are subsequently refined through following layers.

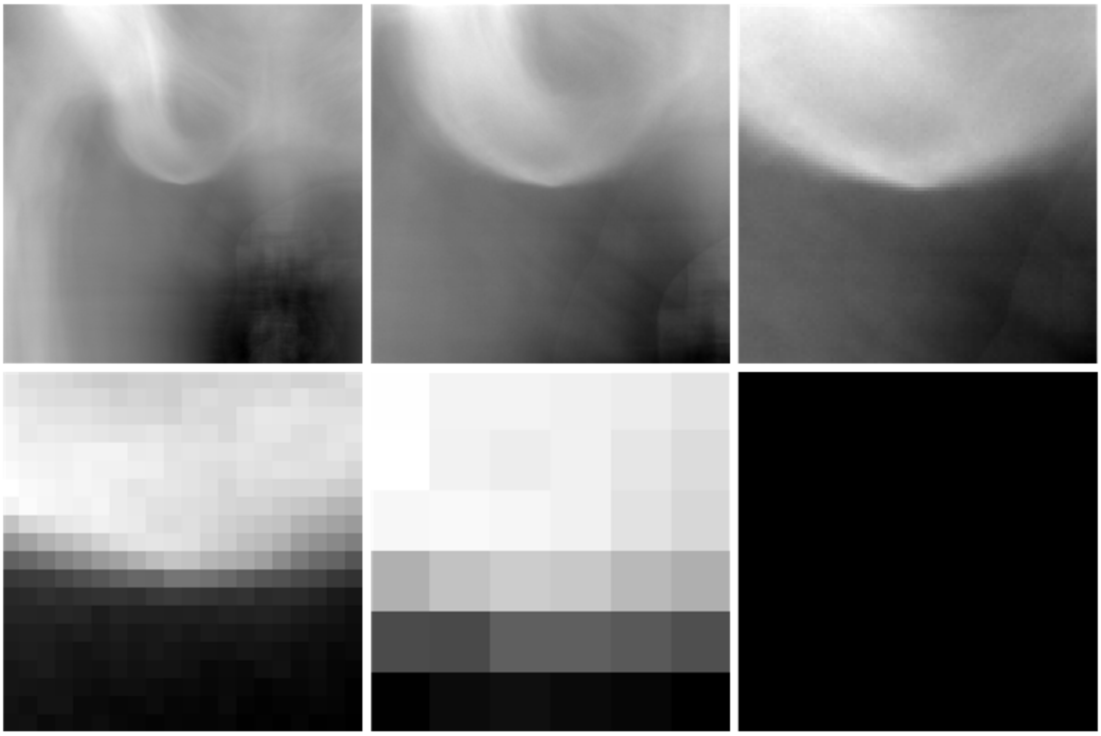

Figure 3.5: Mean image intensities over training set around landmark 1 with reducing s_a from top-left to bottom-right

Since not finding a proper ROI has severe consequences on subsequent calculations (see figure 3.3 right), several ROIs will be spawned controlled by $spawn_a$ based on minima of formula 3.3. Figure 3.3 middle shows another example where deeper layers produce more similar predictions. Hyperparameter $spawn_a$ together with A yield the maximum possible size of $\mathcal{C}_l$ (3.6). It would be reached in the non-redundancy case.

$$|C_l|_{max} = \prod_{a=1..A} spawns_a \tag{3.6}$$

3.3 Convolutional Neural Network Classification

Convolutional Neural Networks (CNNs) are a strong tool for image classification, e.g. for predicting whether a tumor is malignant or benign [32]. In these examples, complete images are fed into a CNN and the output is one of two (binary) or more (multiclass) labels. Landmark detection can also be transformed into a binary classification problem, where each pixel is classified into two categories, *is* (positive class) or *is not* (negative class) a landmark. This said, a CNN is learned and predicts separate for each landmark so that several training sets $\mathcal{D}_{train,l}$ need to be constructed.

CNNs are data hungry, a property that is still unusual to be satisfied in the medical domain with reconsenting/ethical approval issues and data silos. Additionally, for landmark detection, the effort of labeling and the recruitment of domain experts makes data acquisition a resource-intense undertaking. In a previous paper [15], it is shown that even with few images, anatomical landmark detection on skull x-ray data can be accurately solved with CNNs. A reason besides the simple binary problem formulation is that many training data points can be generated out of a single image. This results in a much bigger training set than the image count itself. In comparison to simply taking the SSD as in the RROI algorithm or construct features by hand as in CCP, CNN classification (CNNC) promises to be a better predictor with automatic, data-specific representation learning. It is hence tried out as a third method. What follows is a description of the method applied in [15].

In order to construct $\mathcal{D}_{train,l}$, positive and negative data points are randomly sampled within certain radii as can be seen in figure 3.6 left/middle. Positives representing GT are sampled within a small radius around the true landmark location of each training image. This assumes that nearby pixels can still be considered as valid landmark coordinates which do not decrease performance. A second, bigger circle is spanned as a non-sampling area to provide greater distinction between positive and negative samples. Lastly, a third circle excluding the first two is spanned around the true landmark coordinate to sample negatives. Next, squares are placed with the sample point as their center (see figure 3.6 right). The shuffled combination of subimages $i \in \mathcal{I}_{train}$ from each base image together with their corresponding labels $y_l \in \{0, 1\}$ make up $\mathcal{D}_{train,l}$ and is created for all $l \in \mathcal{L}$.

After dataset collection, the CNNs can be trained landmark-wise. Detailed information about all the mentioned concepts can be found in [33, 34]. Here, only the most important intuitions are given which can be used as repetition. CNNs

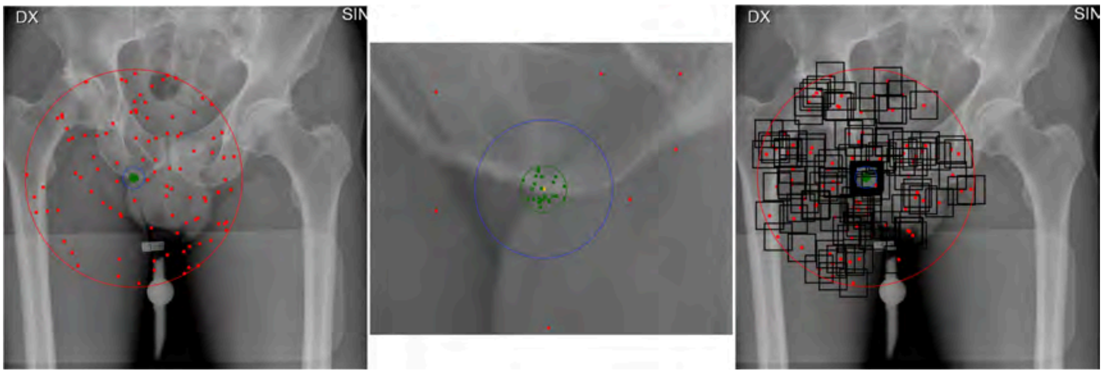

Figure 3.6: Illustrative sampling procedure for landmark 1, left: negative sample center points in red, positives in green, middle: zoomed to true landmark location, non-sampling area represented by blue circle, right: sample boundaries drawn

are compositions of non-linear functions that map an input to an output. They are structured in layers of neurons that are connected to each other. Those connections are parameters, called weights, which are optimized based on the given data, an error tracing method called backpropagation, a learning algorithm like stochastic gradient descent and an objective function.

CNNs in particular consist of convolutional (conv) and most often fully-connected (FC) layers. A conv layer consists again of several feature maps with equal sized 2D shapes. They learn to be responsive to data-specific spatial intensity patterns (features). In the first conv layer, those patterns can be horizontal/vertical/diagonal edges or other pixel arrangements that might not be interpretable by humans. Every feature map comprises neurons. Each neuron is connected to a certain part of the input image (more generally: previous layer) called local receptive field. Neurons react more extensive when exposed to filter (weight matrix) correspondent patterns. Weights are called shared because they build equal valued connections between all local receptive fields and neurons of a specific feature map. This enables a feature map to find the same pattern over the whole image. They therefore account for translations/shifts which occur frequently in the hip dataset.

The next component is the pooling layer which can be interpreted as the second part of a conv layer. A pooling layer reduces the dimensionality and hence overfitting by compressing feature maps. It has no own parameters (weights). The most common and also in this thesis used pooling method is max-pooling which only keeps the maximum values of windows spanned over their associated feature map.

Since shape sizes decrease in deeper layers, receptive fields become bigger relative to what they see. In addition, every feature map of the second (and higher) conv layers has access to all feature maps of the previous layer. The CNN is therefore enabled to combine low-level features like edges to form higher-level representations like e.g. parts of the ischia, trochanters and other pelvis areas. The role of conv layers in a CNN is learning patterns encountered in the training set. FC layers on top of conv layers can combine these arbitrarily to make predictions.

At the beginning of this section, CNNs were defined as non-linear functions. Every neuron's output o is actually only a linear combination of its input intensities x, weights w and bias b. To make them non-linear and better predictors, an activation function like the rectified linear unit (ReLU) is applied on top of the neuron's prior calculation (see formula 3.7).

$$o = max\left(0, \sum_{e \in rec.field} w_e \cdot x_e + b\right) \tag{3.7}$$

Since landmark detection has here been formulated as a binary classification problem, the output is given by a single neuron. Its final value is wrapped between 0 and 1 for which the sigmoid activation function (3.8) is used. A cost function for sigmoid activation that allows fast learning by avoiding small gradients/ weight updates for large output-prediction-differences is binary cross entropy CE (3.9), with $y, \hat{y}$ as GT and prediction over n training instances.

$$f(x) = \frac{1}{1 + e^{-x}} \tag{3.8}$$

$$CE = -\frac{1}{n}\sum_{i=1}^{n}(y_i \cdot ln(\hat{y}_i) + (1 - y_i) \cdot ln(1 - \hat{y}_i)) \tag{3.9}$$

A CNN consists of a forward and backward pass. The forward pass can be seen as conducting step, calculating predictions given inputs. The backward pass via backpropagation is, by intuition, a reflection of each neuron in the network about its share of prediction error that it caused. This reflection makes them adapt in a way such that the prediction is more similar to GT averaged over (parts of) the training set.

Since ideally, every pixel is evaluated in the prediction stage, several pixels might be declared as positive ($\hat{y} > 0.5$). There are two reasons for this: First, subimages defined by nearby pixels around the true landmark location are almost identical. Second, the visual appearance around the true landmark location might occur similarly in other image parts. This is the same problem described in section 3.1 and based on settings also in section 3.2 where we only look at a narrow area around GT. To save computation time, the source image size has been reduced in [15]. Another option is to apply a stride to the original image. This results in evaluating fewer pixels ($\lceil \frac{\sqrt{\#pixels}}{stride} \rceil^2$ if image is quadratic) but can cause performance loss. Figure 3.7 shows a typical classification of the 50 highest estimated coordinates per landmark.

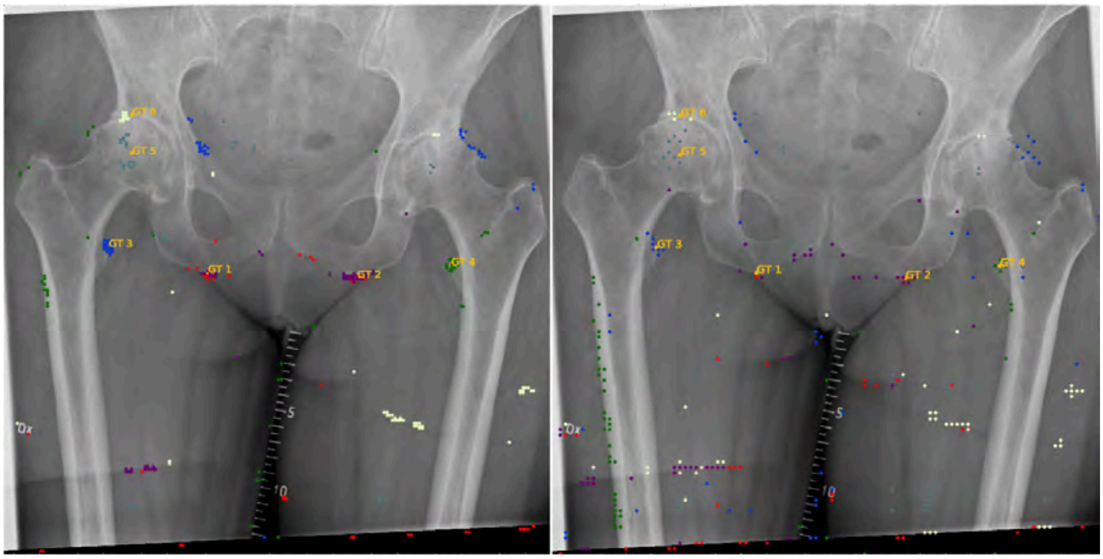

Figure 3.7: Typical CNN classification output with 50 candidates for each landmark 1 to 6 with colors red, purple, blue, green, light blue, white (best viewed digital and/or colored); left: $stride = 10px$, right: $stride = 20px$ results in higher spread

Displaying many outputs means that some might have a low probability value assigned by the network. The candidates build clusters near GTs but also around landmark similar image features as shown in figure 3.8. In this zoomed version, lower numbers represent higher probabilities. Candidates 1, 2 and 3 are all near

GT. However, candidate 5 lies in a different area. All estimates up to number 10 for this image have probabilities of more than 0.99 being landmark 3. More drastic confusions occur between landmarks 1 and 2. Averaging or choosing the coordinate with highest score as final prediction is hence not suitable. How to decide this is the topic of the next subchapter.

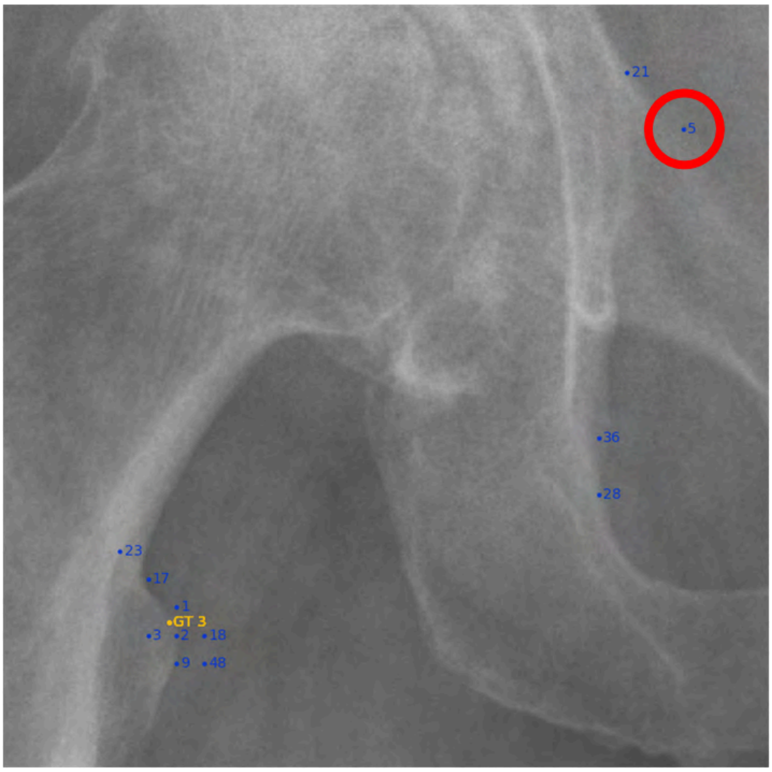

Figure 3.8: Zoomed and ranked version of figure 3.7 right, candidate 5 has erroneously a high probability of representing GT 3

3.4 Shape Model

From all previous methods (section 3.1-3.3), we receive a number of candidates per landmark. Now, a decision needs to be made which candidate to consider as final output. While the number of candidates can be controlled with CNNC such that all landmarks have an equal quantity of suggestions, CCP and RROI do not possess this property. We need to determine which candidate combinations make most sense given spatial relationships (distances, angles) between them including the respective candidate intensity appearance likelihoods. Besides classification

CNN for ALD, the authors of [16, 35, 15] make another contribution by introducing a shape model and a game-theoretic framework to solve this problem. These methods will be described shortly in the following. For more details, refer to the original publications.

The first step is to compute an intensity appearance likelihood map a_l for $l \in \mathcal{L}$ and $c \in \mathcal{C}_l$ (3.10) in test image i_{test}. Fraction $\frac{i_{test}(n)-\mu_{\mathcal{N}_c}}{\sigma_{\mathcal{N}_c}}$ describes the intensity normalized pixel n from neighborhood $\mathcal{N}_c$. The measures μ_l, σ_l are mean and standard deviation matrices built from normalized patches of same size as $\mathcal{N}_c$ centered at l over $\mathcal{D}_{train}$.

$$a_l(c) = \sum_{n \in \mathcal{N}_c} \frac{1}{\sigma_l(n)} \exp\left(-\frac{\left(\frac{i_{test}(n)-\mu_{\mathcal{N}_c}}{\sigma_{\mathcal{N}_c}} - \mu_l(n)\right)^2}{2\sigma_l(n)^2}\right) \tag{3.10}$$

It is calculated with Gaussian kernel density estimation (KDE). KDE can generally and intuitively be described as adding up the likelihood landscape of a variable compared between training set and test instance. Instead of using it for detection as e.g. in [16], it is here taken as part of an evaluator. It gives information about the degree to which a candidate can be considered a respective landmark based on its neighborhood pixels $n \in \mathcal{N}_c$ within an image. A standardization between $[0, 1]$ follows to get probabilities. A classification CNN already outputs probabilities, such that (3.10) must only be applied to candidates received from CCP and RROI.

The next concept is shape likelihood maps s_{l_p,l_q}, computed for all candidates of each possible landmark pair (l_p, l_q) with $l_p \neq l_q$ in a test image. They are constructed with (3.11) as combination of distance D (3.12) and angle Φ (3.13) KDEs comparing test image measures to all training image measures. Hyperparameter λ controls the importance of distance and angle information (set to 0.5) while Δ and Θ (initially 1 and 0) set scaling and rotation changes of a test image's preliminary candidate combination compared to the means in the training set. Standard deviations $\sigma_{D_{l_p,l_q}}, \sigma_{\varphi_{l_p,l_q}}$ are computed on D_{train}. The results of 3.11 are, as in 3.10, standardized.

$$s_{l_p,l_q}(d_{c_{l_p},c_{l_q}}, \varphi_{c_{l_p},c_{l_q}}, \Delta, \Theta) = \lambda D_{l_p,l_q}(\Delta d_{c_{l_p},c_{l_q}}) + (1-\lambda)\Phi_{l_p,l_q}(\varphi_{c_{l_p},c_{l_q}} + \Theta) \tag{3.11}$$

$$D_{l_p,l_q}(d_{c_{l_p},c_{l_q}}) = \sum_{i \in \mathcal{I}_{train}} \frac{1}{\sigma_{D_{l_p,l_q}}} \exp\left(-\frac{(d_{l_p,l_q}(i) - d_{c_{l_p},c_{l_q}})^2}{2\sigma^2_{D_{l_p,l_q}}}\right) \tag{3.12}$$

$$\Phi_{l_p,l_q}(\varphi_{c_{l_p},c_{l_q}}) = \sum_{i \in \mathcal{I}_{train}} \frac{1}{\sigma_{\varphi_{l_p,l_q}}} \exp\left(-\frac{(\varphi_{l_p,l_q}(i) - \varphi_{c_{l_p},c_{l_q}})^2}{2\sigma^2_{\varphi_{l_p,l_q}}}\right) \tag{3.13}$$

With both appearance and shape values, partial payoff matrices W_{l_p,l_q} are calculated (3.14) for each landmark pair l_p, l_q. They hold likelihoods for all candidates c_{l_p} being the respective landmark l_p when encountered with each candidate c_{l_q} in a test image. Both appearance likelihood of a c_{l_p} and the shape likelihood between each of the candidates of two distinct landmarks are considered. The importance of both is controlled by τ and is here set to 0.7. Shape information is factored in to a higher rate because candidate predictions are more error-prone due to image symmetries, local view and previously discussed hyperparameters like stride.

$$W_{l_p,l_q}(c_{l_p}, c_{l_q}, \Delta, \Theta) = (1-\tau) \cdot a_l(c_{l_p}) + \tau \cdot s_{l_p,l_q}(d_{c_{l_p},c_{l_q}}, \varphi_{c_{l_p},c_{l_q}}, \Delta, \Theta) \tag{3.14}$$

With 3.15, the optimal candidate combination is determined as sum of landmark candidate payoffs w. Function $h(x) = \frac{x}{x+\alpha}$ with α corresponding to the average payoff, prefers combinations with more similar outcomes than strong-weak arrangements. The latter would force one landmark be represented by a weak candidate if a candidate of another landmark has outstandingly high likelihoods. The authors describe this concept as equalization.

$$\psi(\mathcal{L}, \mathcal{C}, W, \Delta, \Theta) = \underset{w}{\operatorname{argmax}} \left(\sum_{l_p \in \mathcal{L}} h \left(\sum_{\substack{l_q \in \mathcal{L} \\ l_p \neq l_q}} (W_{l_p,l_q}(c_{l_p}, c_{l_q}, \Delta, \Theta)) \right) \right) \tag{3.15}$$

The resulting candidate combination can be reevaluated (formula 3.15) with Δ, Θ being adjusted by comparing rotation and scale of the current best combination with the training set average. Finally, random forest regression is applied repeatedly by leave-one-out principle. The corresponding candidate nearest to this prediction is considered as the final estimate and will help to eventually refine estimates for the other landmarks. For both procedures, candidates are included from the complete pool $\mathcal{C}$.

3.5 Cascaded Convolutional Neural Network Regression

The last tested method is regression using CNNs. Regression constitutes continuous output. In the medical domain, examples are prediction of care expenses, a person's age when contracting a particular illness based on lifestyle and preconditions or the hospital return rate of patients with chronic diseases. Since landmarks are represented as continuous pixel coordinates, regression can be applied, too.

The input in such a network is the base image itself. The training set must thus not be constructed out of subimages. It is a disadvantage that the training set size is restricted to these input images which is much smaller compared to section 3.3. The chance of overfitting increases accordingly. The output, in contrast to the binary decision in the classification chapter, will encompass the axis coordinates for each landmark, resulting in 12 continuous outputs for 6 landmarks. As cost function, mean squared error (MSE) is used (general form see 3.16). Besides in- and output, the configuration is similar to the classification network.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{3.16}$$

In prediction phase, a classification network needs to evaluate sub images for each pixel and landmark. A regression network though only processes a single image of original size as during training. However, the task at hand is more difficult because the output range is much broader. Furthermore, a classification network must be learned for each landmark. This makes learning exhausting for detecting many landmarks which is e.g. needed for subsequent segmentation. In contrast, only a single regression network is leveraged irrespective of landmark quantity.

A nice property of modeling all landmarks with a single network is that they learn together, i.e. landmarks influence each other while they train. This is the wanted effect of the complex shape model (section 3.4) but comes free and implicit with this architecture choice. The reason why mutual influencing works can be justified by the dependency of weight gradients on equal neuron inputs in the last layer. Modeling related tasks in parallel with a shared representation to support the training of individual tasks is called multi-task learning [36].

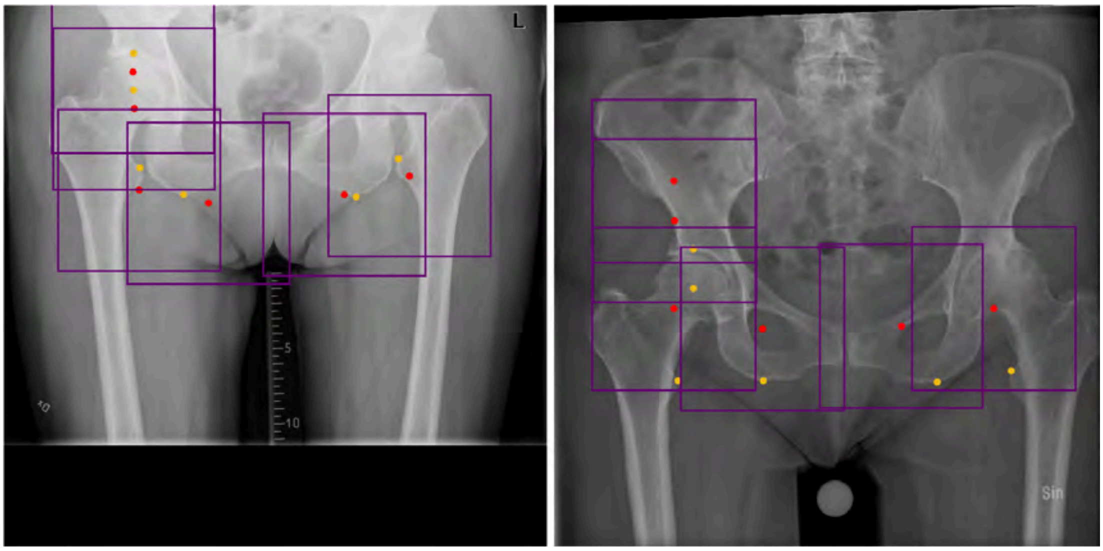

Figure 3.9: First regression prediction (red) vs. truth (gold) with ROI bounding boxes centered at output coordinates, left: typical scenario on a validation image, right: uncommon output on a validation image

Figure 3.9 depicts a typical and a rare prediction on two validation images. The predictions are far off from true locations. There are three reasons for this behavior:

1. Too few training images are available.
2. Input images have a high resolution causing overfitting.
3. The output format is continuous which is harder to fit than binary. In addition, compared to classification in which the combination of high-level features make up the decision for a class, the intuition for regression is not easy to restore.

It is a valuable property of multi-task learning to model landmark positions not only from image patterns but also based on adjacent landmarks. It causes the network to preserve shape. If e.g. 1 landmark were not located near GT or the distance differed significantly from other landmarks, we would not be able to consider subsequent improvements. A CNN regression network lays hence the basis for creating regions of interest to reduce dimensionality. On these ROIs, additional regression CNNs with smaller input can be applied repeatedly. Already in [23], the authors created several CNNs in row, called cascades, to make coarse to fine-grained landmark predictions on a big data set of small (39x39 px) face images. In their paper, even the first network makes reasonable guesses and only minor gains on following CNNs can be observed due to smaller image size.

With cascades, we loose the property of learning a single network. In fact, networks for each landmark must be trained after the first regression. New training and validation sets need to be composed. An important hyperparameter is the input image (ROI) size s_{ROI} of every cascade level. One way to determine it is to look more closely at the distances between prediction and true positions on validation data. More specifically, we plot these distances together with the rate of GTs that lie within them for successive regressions in the cascade (figure 3.10). The function's steepness increases with the number of regressions meaning that more true locations are in the prediction's vicinity. The likelier functions become (CCNNR2 & 3) the fewer improvements can be expected.

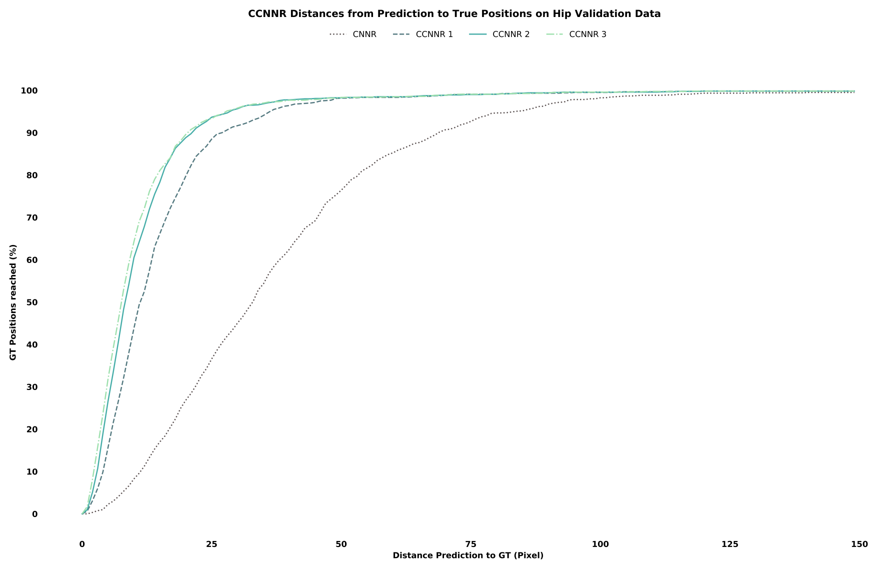

Figure 3.10: Distances from prediction to GT for all regressions in the cascade

Ideally, we favor a ROI size that ensures the inclusion of all GTs. At least in validation data, one can guarantee for it by taking the maximum euclidean distance $d_{l\hat{l}max}$ between ground truth l and prediction $\hat{l}$ (3.17). If $d_{l\hat{l}max}$ is far away from a distance comprising e.g. 99% of true coordinates (e.g. for CNNR, it is beyond 150 px), then it is sensible to neglect the noise in favor for a smaller region size.

$$d_{l\hat{l}max} = \max_{i \in \mathcal{I}_{val}} \sqrt{(l_x - \hat{l}_x)^2 + (l_y - \hat{l}_y)^2} \tag{3.17}$$

A safety factor d_s of desired value is added (3.18) because test data can differ from previous considerations and to leave some space for better detection of landmarks based on its surrounding. This in fact results only in half the size of a ROI since GTs can be located in any direction. s_{ROI} as in (3.19) adds 1 to have a real center.

$$d_h = d_{l\hat{l}max} + d_s \tag{3.18}$$

$$s_{ROI} = 2d_h + 1 \tag{3.19}$$

In order to train a new CNN to find landmarks within ROIs, a new training set is created per landmark. One acquisition option is to create subimages (of size s_{ROI}) around GT positions once for every training image. Another better option is to augment the data by random shifts from GT within some range $[0, shift_{max}]$. Several ROI images are sampled per base image to create a much bigger training set to reduce overfitting and hence create a stabler predictor. This is reasonable since the random shifts provide repeatedly not only altered (as any affine transformation does), but new pattern information.

This introduces another hyperparameter called padding p which restricts the shift range. As can be seen in figure 3.10, the distance difference between e.g. 90% and 100% of GTs reached is reasonably high. If we were to allow shifts of size d_h, the built training set would not represent validation data well because most prediction to GT distances are far below this value. In addition, features in the neighborhood necessary for accurate model development will not be visible anymore. Padding ensures that ROIs still include enough space around the GT to learn the underlying structure for solving the problem. Therefore, we reduce d_h by p to get the maximum allowed shift $shift_{max}$ (3.20).

$$shift_{max} = d_h - p \tag{3.20}$$

Figure 3.11 left shows an example of these values. Figure 3.11 right is an example of a newly generated training image for landmark 3 with almost $shift_{max}$.

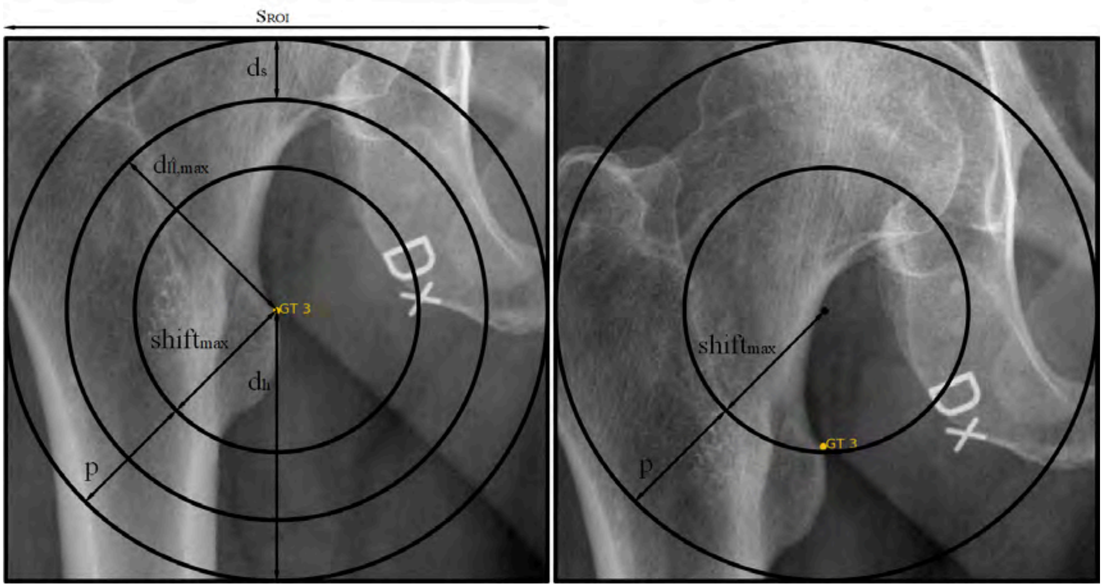

Figure 3.11: Constructing a new training set for landmark 3 based on first regression results, left: ROI measures based on validation data, right: randomly shifted sample for next regression dataset

Along the cascade, the number of layers and feature maps can decrease since the important features become low-level. A remarkable property of CNN cascades is that even with few training data, good results can be achieved by dividing the problem into two tasks. Depending on the image size and data available, the first CNNs create ROIs to simplify the problem while the backmost CNNs are responsible for accurate prediction. Figure 3.12 shows example prediction paths of a complete image and has been chosen since it visualizes continuous improvement well. More predictions on different case types can be found in appendix B. Figure 3.13 shows the prediction path of a single landmark with its corresponding ROIs. On the hip dataset, most improvement can generally be seen between first and direct follow up regression.

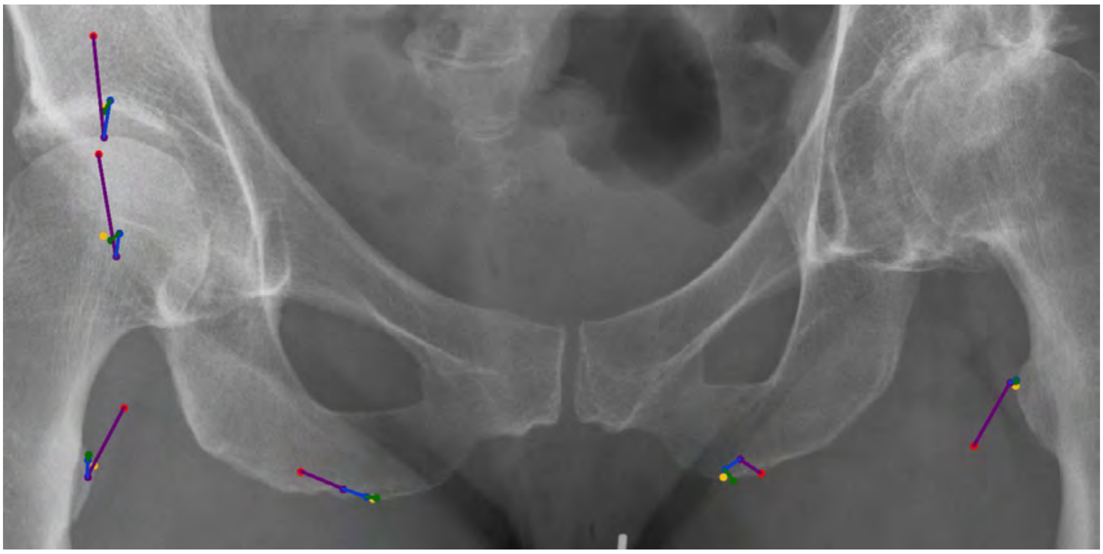

Figure 3.12: Regression paths example with more accurate outcomes along the cascade, red=1st estimate, purple=2nd, blue=3rd, green=final, gold=true position

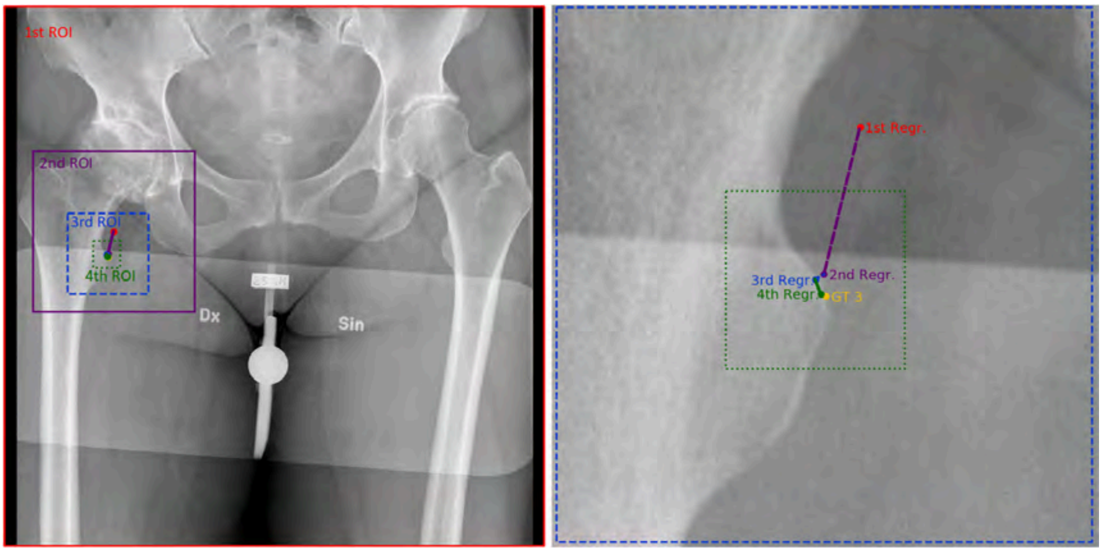

Figure 3.13: Regression paths for landmark 3, left: ROI areas, right: Zoomed into third ROI, enumerating regression outputs

4 Data Processing

The hip dataset has been extracted under ethical approval number 2017-444-31 with anonymization according to established practices pursuant to the Swedish data protection act (Personuppgiftslagen). Anonymized image data was obtained from the Linköping university hospital, a Sectra PACS and orthopedic package customer.

The dataset consists of around 4000 postoperative and preoperative cases (with and without prostheses) of both genders and are stored in DICOM files. Postoperative images were excluded because hip replacement surgery planning with the orthopedic package is mostly conducted for preoperative cases. Furthermore, it simplifies the landmark detection task. The data is raw, i.e. images might be duplicates and are not filtered to always have clear/unambiguous landmark positions. Besides duplicates, only x-rays in which guided manual planning would not have been technically possible (e.g. landmark concealed/not in image boundaries) have been excluded. Due to time limitations, a dataset of size 1054 remained post removal. Many outliers are part of the kept set that make landmark placement challenging even for humans. Those high variances originate from:

- bad contrast/illumination (e.g. too high consistent brightness)
- patient gender and physics
- arthritis (caput center and acetabular roof merely or not visible)
- flat ischia (placement of landmarks ambiguous without contour changes)
- invisible trochanters due to patient rotation
- artifacts (words, objects of unknown type)

Appendix A shows some examples of these conditions.

DICOM files possess the property *Photometric Interpretation* that specifies the intended perception of pixels and is changed to have equal value (0 = *black*). Some images were upside-down and needed a rotation.

Labels are not included and were obtained with the help of employees of Sectra's orthopedics department acting as experts. Therefore, a labeling tool has been developed. The design goal was to resemble the click guide process. This includes color choice, click order and visual aids like points representing landmarks, the line between right/left ischium as well as a circle to facilitate capturing the caput (see figure 4.1). Furthermore, a right click reverts selections to allow changes and a console provides an overview of the current state. Packages of 10 files each were stored at the network file system and every expert could work on these depending on available time. All experts were educated about the conduct and landmark definitions (see chapter 1) prior to labeling. Obtained landmarks were reinspected and obvious errors corrected.

Figure 4.1: Label clicker: every click draws a new point

The files have varying number of pixels, pixel ratios as well as spacings (mm per pixel). Data modeled by a standard convolutional neural network need an equal number of input dimensions while CCP and RROI do not. In order to avoid the creation of different datasets, the original will be altered to have files with equal pixel quantities (neural network input dimensions). Therefore, the format is first changed to TIFF for better access to image processing libraries and easier file handling. For ratio equalization (1:1), the shorter image side is zero-padded at its upper extremum to match the longer side. Thereafter, images and landmark coordinates are resized to the smallest image (2021 pixels) in the set for pixel equalization. Images are lastly intensity range adjusted [0-1] to account e.g. for varying brightness and because the original files have differing minimum and maximum intensities (DICOM tag: *Bits Allocated*).

While many training instances are available for CNNC due to multisampling within a single image, the quantity of training examples for all other techniques is restricted by dataset size. With data augmentation, the training data is expanded by small random changes of the original data which results in more robust models that better handle overfitting. Therefore, every training image (and its corresponding set of landmarks) is augmented several times and equally often by translation and rotation which results in 3000 training images (4 augmented images per original + original). Rotations up to $4°$ are allowed in any direction with exception of $0°$. Similarly, maximum translations of 8% of the image dimension are allowed. The offset is filled with zeros. The bounds were chosen to still represent realistic cases. If the resulting landmarks lie outside the image dimensions, a new random constellation is computed until they fit. For every part of the cascade, 6000 training and 2000 validation images were generated (10 augmentations per original).

The dataset is split into training, validation and test set consisting of 600, 200 and 254 images, respectively. Models consisting of parameters (weights in the neural network case) are built from the training set. Hyperparameters (values set manually affecting the model performance) are chosen based on the validation set. Evaluation is done on the test set.

5 Implementation

Python 3.5 is chosen as general programming language allowing fast experimentation. In addition, many third-party packages can be leveraged. Keras with Tensorflow backend is used for neural network implementations. A Nvidia GeForce GTX 1060 6GB has been provided for faster neural network learning. In this chapter, implementation details in form of pseudocodes are given for CCP and RROI. Furthermore, applied CNN architectures are depicted. Hyperparameters are mentioned in both subchapters.

5.1 Pseudocodes

5.1.1 CCP Feature Extraction

OpenCV implementations are utilized to preprocess images (histogram equalization, Gaussian blurring, canny, closing). Gaussian blurring is exerted with kernel size $(5, 5)$. Standard deviations are calculated from the kernel. Canny thresholds are $t_{min} = 0.7$ and $t_{max} = 0.3$ and a Sobel kernel shape $(5, 5)$ is chosen. Morphological closing is implemented with a $(5, 5)$ kernel, too. Finally, scikit-image's implementation for skeletonization [37] is harnessed.

Figure 5.1 shows the learning process pseudocode of CCP which is merely the gathering of feature objects in a landmark dictionary. Several hyperparameter settings have been tested with $t_c = 15$ px, $r_{\mathcal{N}} = 15$ px and $k = 1$ (deactivating all other variation settings) achieving best results. Contours as a collection of change coordinates (in the sense of every direction change) are calculated with OpenCV's implementation of [38]. The *calcChains* function returns a list of chains as numerical strings and their respective start coordinates for tracking. It does this by finding a starting point from each contour's coordinates and traversing it with binary filtering. Already crossed points are added to a blacklist. New chains are initialized when passing junctions. Based on chains, CPs composed of type and coordinate are extracted. Finally, features are calculated from CPs occurring near the training images' landmarks.

Init FDB *as empty mapping*
for all $(i, \mathcal{L}_i) \in \mathcal{D}_{train}$ **do**
    $i = preprocess(i)$
    $contours = findContours(i)$
    $chains, startPoints = calcChains(contours)$
    $CPs = calcCPs(chains, startPoints)$
    $FDB := calcFeatures(CPs, \mathcal{L}_i)$
end for

Figure 5.1: CCP learning procedure

The process of calculating changes is depicted in figure 5.2. If a chain string is too short, it will be ignored. The threshold is here set to $2t_c$ with at least $1t_c$ needed to find a starting direction and another $1t_c$ to obtain a target direction at the maximum. The initial starting direction $cpFirst$ is calculated with function $findInitDominantDirec$ which counts CC occurrences and outputs the one with highest count as soon as t_c is reached for that particular CC. In case t_c is not passed after full traversal, no CP is found for this chain.

Variables $changeContribs$ and $firstCodeCoords$ store the number of occurrences and the coordinate of first occurrence for every CC within extraction of a CP. During chain string traversal, these variables are updated. Variable $sumOfChanges$ representing the change magnitude is incremented/decremented based on the current CC and its compass step distance to $cpFirst$. On surpassing t_c, the target direction $cpSecond$ is determined as CC with most contributions and a new CP is assembled. The starting direction is set to the target direction and all observer variables are reset. This also applies to the case when the threshold becomes lower than 0.

The function $calcFeatures$ in figure 5.3 returns a dictionary that maps from landmarks to lists of features F_i for a particular image i. A change coordinate near a ground truth landmark and its k nearest neighbors compose a feature representing that landmark. The prediction is eventually an easy FDB lookup as depicted in figure 5.4 where line 2 is a shortened version of the previously described functions.

```
function calcCPs(chains, startPoints)
    Init CPs as list
    for all chain ∈ chains, startPoint ∈ startPoints do
        if |chain| < 2t_c then
            nextIteration
        end if
        cpFirst, currentCoord = findInitDominantDirec(chain, startPoint, t_c)
        Init firstCodeCoords, changeContribs as empty/zeroed CC mapping
        Init sumOfChanges as 0
        while traverse(chain) with cc ∈ chain do
            updateObservers(firstCodeCoords, changeContribs, sumOfChanges, cc)
            if sumOfChanges > t_c then
                cpSecond = getMaxContributor(changeContribs)
                type = (cpFirst, cpSecond)
                coord = firstCodeCoords[cpSecond]
                CPs := (type, coord)
                cpFirst = cpSecond
                reset(changeContribs, sumOfChanges, firstCodeCoords)
            else if sumOfChanges < 0 then
                reset(changeContribs, sumOfChanges, firstCodeCoords)
            end if
        end while
    end for
    Return: CPs
end function
```

Figure 5.2: CCP algorithm: Change point calculation

```
function calcFeatures(CPs, L_i)
    Init F_i as empty mapping
    for all cp ∈ CPs do
        for all l_id ∈ L_i do
            if euclDist(cp.coord, l_id) < r_N then
                N = getNeighborChanges(cp.coord, CPs, k)
                F_i[id] := (cp, N)
            end if
        end for
    end for
    Return: F_i
end function
```

Figure 5.3: CCP algorithm: Feature extraction

```
for all i ∈ I_test do
    F_test = getFeatures(i)
    L̂_i = lookup(F_test, FDB)
end for
```

Figure 5.4: CCP algorithm: Prediction as a lookup

5.1.2 RROI Prediction

Figure 5.5 shows the candidate prediction pseudocode for all landmarks of a single test image. Means over $\mathcal{I}_{train}$ with sizes $s_a,\ a = 1..A$ are taken in line 4 and embody model learning. The reference coordinate $refCrd$ corresponds initially to the point of origin (top-left corner) and is later adjusted to the respective ROI starting position in the global coordinate system. Coordinates of all ROI starting positions will be stored in $ROICrds$ as a layer to coordinate list dictionary. Finally, the center positions of the deepest ROI are considered as landmark candidates. If $a = A$ and $s_a = 1$, then candidates correspond to $ROICrds[-1]$.

ROI calculation is done horizontally first with a recursive function $calcROIs$ as depicted in figure 5.6. Variable $patchDiffs$ holds the SSD values of mean and sliding window sampled patches of size s_a, calculated in line 9. Starting coordinates relative to the current search area and of quantity $spawn_a$ from lowest SSD patches are stored in $relCrds$. The resulting global coordinates are stored in $ROICrds[a]$ considering the current reference coordinate. The newly encountered ROIs are used to calculate ROIs one layer deeper until maximum depth A is reached.

The following hyperparameters, found through grid-search, produce best results on validation data:

- $A = 9$
- $s_{1..A} = \{1000, 200, 100, 50, 10, 6, 4, 2, 1\}$
- $stride_{1..A} = \{100, 50, 20, 10, 1, 1, 1, 1, 1\}$
- $spawn_{1..A} = \{3, 7, 2, 1, 1, 1, 1, 1, 1\}$

Notice that hyperparameters are dependent on image size. Generally, the first layers are leveraged to receive distinct rough estimates (high size, stride and spawns) while deep layers become more specific because less errors are made due to similar visual features.

for all $l \in \mathcal{L}_i$ **do**
    Init $refCrd\ as\ (0, 0)$
    Init $ROICrds\ as\ empty\ mapping$
    $means = getMeanPatchesLm(\mathcal{I}_{train}, s_{1..A}, l)$
    $ROICrds := calcROIs(i_{test}, refCrd, ROICrds, means, a = 1)$
    $\mathcal{C}_{i,l} = getLastLayerCenters(ROICrds)$
end for

Figure 5.5: RROI algorithm: Landmark candidate prediction

```
function calcROIs(i, refCrd, ROICrds, means, a)
    if a > A then
        Return: ROICrds
    end if

    Init patchDiffs as empty list
    while slidingWindow(i, s_a, stride_a) do
        patch = generateNext()
        patchDiffs := calcPatchDiff(patch, means[a])
    end while
    relCrds = getMinDiffCoords(patchDiffs, spawn_a)
    for all crd ∈ relCrds do
        ROICrds[a] := refCrd + crd
        refCrd := ROICrds[a]
        refImg = getSubImg(i, crd, s_a)
        ROICrds := calcROIs(refImg, refCrd, ROICrds, means, a + 1)
    end for

    Return: ROICrds
end function
```

Figure 5.6: RROI algorithm: Recursive function for ROI coordinate calculation

5.2 CNN Architectures and Learning

A generic CNN architecture is illustrated in figure 5.7. It gives some intuition about how CNNs learn representations where the first weight matrix learned to recognize vertical, the second diagonal and the third horizontal edges. Feature maps exemplify reactions to the input in an illustrative manner, with black regions decoded as high activations. Receptive field windows of conv 2 view more area of its input due to pooling's compression and have access to all maps from pooling 1 so that higher-level feature compositions are learned. Regarding the amount of layers, a conv and max pooling layer are described as a single conv layer. The max-pooling window width and height are chosen to be $(2, 2)$. All CNNs possess 2 FC layers, the first having 512 neurons. All hidden layers are ReLU activated.

All networks are learned with Adam and standard parameters from the original paper [39].

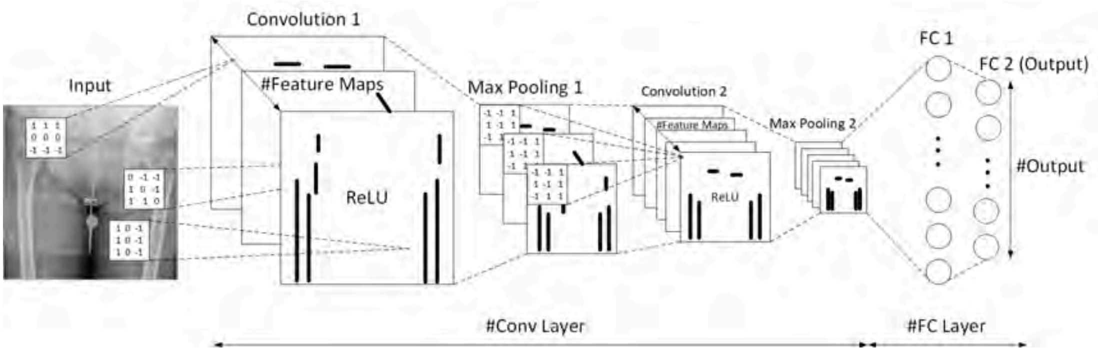

Figure 5.7: Generic CNN architecture with intuition of low to high-level feature learning

Further settings as comparison between all CNN variants are listed in table 5.1. Different settings for $\#Conv$, $\#FeatureMaps$, $Rec.Field$ and $Regularization$ were tuned with a reduced training/validation set and a small amount of epochs. The basis image size for sampling in CNNC is kept at 2021 x 2021 px with $stride = 10$ px and $radii = (10, 30, 550)$ px. For CCNNRs, input dimensions are chosen based on the heuristic presented in chapter 3.5 with $d_h = 250, 150, 50$ px and $shift_{max} = 100, 50, 30$ px for the 3 refinement CNNs.

The mini-batch size for CNNC is set to 64 due to its smaller input dimensions with batch normalization after every hidden activation. The choice of batch size is especially restricted for CNNR due to high input dimension, choice of architecture and limited GPU memory. For that reason, a batch size of 1 must be applied. Within the cascade (CCNNRs), it is consistently increased to 10 to have less hyperparameters to optimize. For the same reason, architecture depth and feature map quantities are restricted. Increasing them for CNNR and CCNNR 1 (e.g. with more GPUs/GPU memory) might further improve results due to the outcome's dependency on high-level features.

The number of feature maps increases for deeper layers as many more complex combinations of previous features can be composed to account for high-level variations (edge vs. pelvis). Receptive fields are reduced over layers as their area of view increase. Zero-padding is applied to equalize output and input maps with one exception. In conv 1, it is unlikely to neglect image structures near landmarks at the boundaries, so that we can use larger receptive fields to shrink dimensionality in following layers. Strides are set such that the output shape shrinks consistently and the number of overall parameters is kept small within the FC layers. For binary classification, sigmoid is a proper output activation function, with a single last layer neuron. ReLU can be applied for continuous landmark coordinates ≥ 0 for all $2|\mathcal{L}|$ neurons in the multi-task setting and only 2 outputs for cascaded networks.

	CNN Variants Hip Dataset				
Settings	CNNC	CNNR	CCNNR 1	CCNNR 2	CCNNR 3
$Input\ Dim.$	111x111	2021x2021	501x501	301x301	101x101
$\|Batch\|$	64	1	10	10	10
$Norm.$	yes	no	yes	yes	yes
$\#Conv$	3	4	4	4	3
$\#FC$	2	2	2	2	2
$\#Feature$ $Maps$	$64, 128$ 192	$64, 128$ $192, 256$	$64, 96$ $128, 160$	$64, 96$ $128, 160$	$64, 128$ 192
$Rec.Field$	$(5,5), (5,5),$ $(3,3)$	$(9,9), (5,5),$ $(5,5), (3,3)$	$(9,9), (5,5),$ $(5,5), (3,3)$	$(5,5), (5,5),$ $(5,5), (3,3)$	$(5,5), (5,5),$ $(3,3)$
$Padding$	yes	$Conv1 : no$ $Rest : yes$	yes	yes	yes
$Pooling$	$Max(2,2)$	$Max(2,2)$	$Max(2,2)$	$Max(2,2)$	$Max(2,2)$
$Strides$	$2,1,1$	$4,2,1,1$	$2,2,1,1$	$2,1,1,1$	$2,1,1$
$\#Output$	1	12	2	2	2
$Output$ $activation$	$Sigmoid$	$ReLU$	$ReLU$	$ReLU$	$ReLU$
$Loss$	$bin.cross-$ $entropy$	MSE	MSE	MSE	MSE
$Regulari-$ $zation$	$L2(0.01),$ $DO(0.5)$	$L2(0.01)$	$L2(0.01),$ $DO(0.5)$	$L2(0.01),$ $DO(0.5)$	$L2(0.01),$ $DO(0.5)$

Table 5.1: Architecture Setting for all CNN variants using the hip dataset

6 Evaluation

In the first subchapter, all used metrics are defined. Thereafter, all results are mentioned for both hip and skull datasets. Further reasoning and explanations are given in chapter 7.

6.1 Metrics

Two general and two application specific metrics are calculated for method comparison over $\mathcal{D}_{test}$ with corresponding predictions $\hat{\mathcal{L}}$. The first general metric is the mean accuracy within certain distance bounds $MADB$ (6.3). It uses the euclidean distance (6.1) with Δ describing the difference of the specified image dimension as well as a counter cnt (6.2) which always returns 1 when $\hat{l}$ lies in distance bound b with $b \in bounds = \{2, 3.5, 5, 7.5, 10\}$ mm.

$$ED(l, \hat{l}) = \sqrt{\Delta x_{l,\hat{l}}^2 + \Delta y_{l,\hat{l}}^2} \tag{6.1}$$

$$cnt(x) = \begin{cases} 0 & \text{if } x > 0 \\ 1 & \text{if } x \leq 0 \end{cases} \tag{6.2}$$

$$MADB = \frac{\sum_{l_p, \hat{l}_p \in \mathcal{L}, \hat{\mathcal{L}}} \sum_{i \in \mathcal{I}_{test}} cnt(ED(l_p^{(i)}, \hat{l}_p^{(i)}) - b)}{|\mathcal{I}_{test}| \cdot |\mathcal{L}|} \tag{6.3}$$

The second metric is the mean radial error MRE (6.4) and sums over distances itself.

$$MRE = \frac{\sum_{l_p,\hat{l}_p \in \mathcal{L},\hat{\mathcal{L}}} \sum_{i \in \mathcal{I}_{test}} ED(l_p^{(i)}, \hat{l}_p^{(i)})}{|\mathcal{I}_{test}| \cdot |\mathcal{L}|} \tag{6.4}$$

The first application specific metric is the mean leg length discrepancy error $MLLDE$ (6.7) with LLD (6.6) as difference between perpendicular distances d of trochanter landmarks $l_t \in \{l_3, l_4\}$ from the ischium line defined by l_1, l_2 (6.5).

$$d(l_1, l_2, l_t) = \frac{|\Delta y_{l_2,l_1} \cdot l_{t,x} - \Delta x_{l_2,l_1} \cdot l_{t,y} + l_{2,x} \cdot l_{1,y} - l_{2,y} \cdot l_{1,x}|}{ED(l_1, l_2)} \tag{6.5}$$

$$LLD(L) = |d(l_1, l_2, l_3) - d(l_1, l_2, l_4)|, \;\; l_1 \; to \; l_4 \in L \tag{6.6}$$

$$MLLDE = \frac{\sum_{i \in \mathcal{I}_{test}} |LLD(\mathcal{L}) - LLD(\hat{\mathcal{L}})|}{|\mathcal{I}_{test}|} \tag{6.7}$$

Finally, the mean ischium line angle difference $MIAE$ (6.8) is measured. It gives information about the correctness for landmark combination estimations l_1, l_2 that define both orientation of the cup and are part of LLD calculation. It is measured based upon angles $\angle l_1 l_2 \leq 180°$ here starting from a horizontal line right to l_2 with a counterclockwise direction.

$$MIAE = \frac{\sum_{i \in \mathcal{I}_{test}} |\angle l_1^{(i)} l_2^{(i)} - \angle \hat{l}_1^{(i)} \hat{l}_2^{(i)}|}{|\mathcal{I}_{test}|} \tag{6.8}$$

6.2 Results

At the beginning of this project, only 143 images were available. Those are leveraged for determining inter-expert MRE and to address two important issues. First, to get a feeling about human performance on this task. Second, to estimate how accurate the system can be approximately based on the later received training data. Two rounds with the setting described in chapter 4 are conducted with experts labeling an image collection in the first round and taking another from a distinct expert in the second. The MREs in mm for landmarks 1 to 6 are 4.76, 4.07, 1.49, 1.81, 2.99, 2.7, for all landmarks 2.97 ± 3.24. At that time, the labeling tool did not include an ischium line helper which could cause higher errors encountered for the first two landmarks. These results should be interpreted as an indicator for the two questions raised above and by no means as comprehensive answers because the sample size is too small. Figure 6.1 shows the landmark-wise MRE while figure 6.2 exemplifies used bounds for MADB on a hip image.

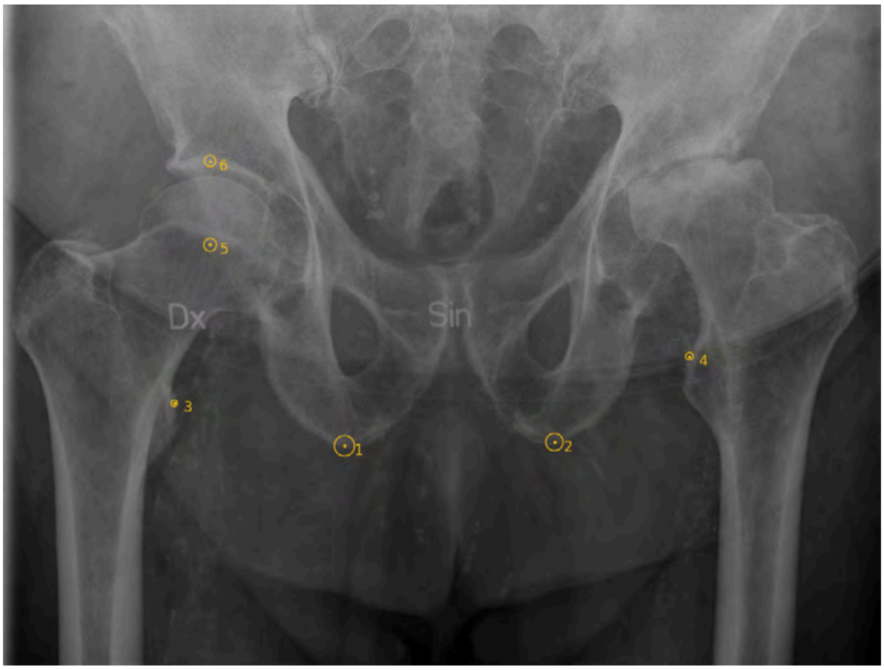

Figure 6.1: MRE for all landmarks compared between experts on 143 images.

In figures 6.3 and 6.4, metric MADB is displayed for all 6 and first 4 landmarks. Method CCP is only considered for the latter plot since it will not always find candidates for landmarks 5 and 6. It was not designed to handle landmarks distant from edges. Another reason for separating plots is that the main task is set to find the first 4 landmarks only, due to the subjectiveness of placing cups. For CNNC, it's worth to mention theoretical possible results considering candidates nearest to GTs as final estimations because large differences to figure 6.3 with a stride of 10px were encountered. MADB in % for distance bounds 2 to 10 mm are 75.8, 85.8, 89.3, 91.5 and 92.6.

Table 6.1 lists the MRE for every landmark, additionally with standard deviations for 4 and 6 groups. Figure 6.6 and 6.7 show the application metrics separated by performance (be aware of scales). For the best performing method CCNNR with 3 successive refinements after the initial regression, histograms for LLD and

Figure 6.2: Used distance bounds (2, 3.5, 5, 7.5, 10) in mm on example

ischium angle error are depicted in figures 6.7 and 6.8.

	Landmarks							
Algorithm	1	2	3	4	5	6	first 4	all
CCP	49.70	42.81	48.82	29.83	–	–	42.79 ± 35.61	–
$RROI$	24.83	27.00	20.51	33.68	23.64	32.64	26.51 ± 39.24	27.05 ± 41.45
$CNNC$	7.26	10.00	12.84	19.61	12.72	19.60	12.43 ± 29.59	13.67 ± 32.39
$CNNR$	9.22	10.07	8.60	9.21	7.33	7.54	9.27 ± 5.47	8.66 ± 5.23
$CCNNR\,1$	3.76	3.87	2.53	2.56	2.98	3.52	3.18 ± 2.98	3.20 ± 2.80
$CCNNR\,2$	2.54	3.16	1.79	1.69	2.81	2.83	2.30 ± 2.73	2.47 ± 2.52
$CCNNR\,3$	2.46	2.80	1.59	1.60	3.06	2.51	2.12 ± 2.70	2.34 ± 2.50

Table 6.1: MRE for all algorithms, reported on all landmarks

Finally, CCNNRs are also evaluated on the IEEE ISBI 2015 Challenge dataset [1] for finding 19 skull landmarks needed for automated quantitative cephalometry. It is compared to the previously best performing methods as in [15]. The MADB results are shown in 6.9 and 6.10 for two different test sets in the bounds 2, 2.5, 3 and 4 mm. MRE in mm for test set 1 is 1.51 ± 1.29, for test set 2 1.79 ± 1.08. More information about the data and implementation as well as example images can be found in appendix C.

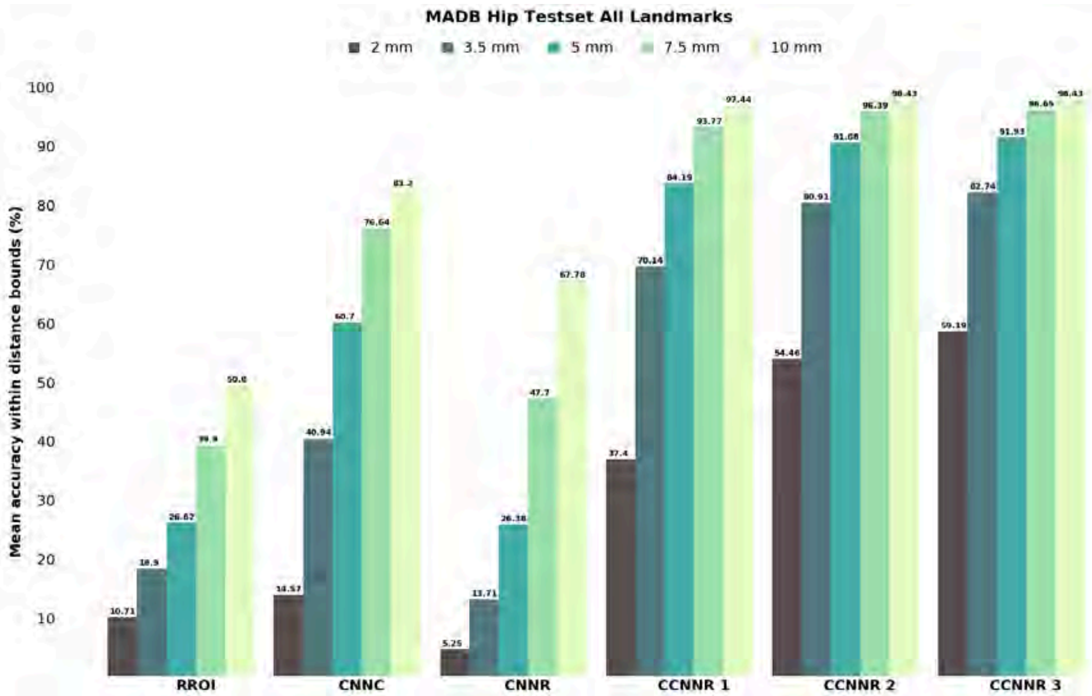

Figure 6.3: MADB on test set including all landmarks, reported on all methods except CCP

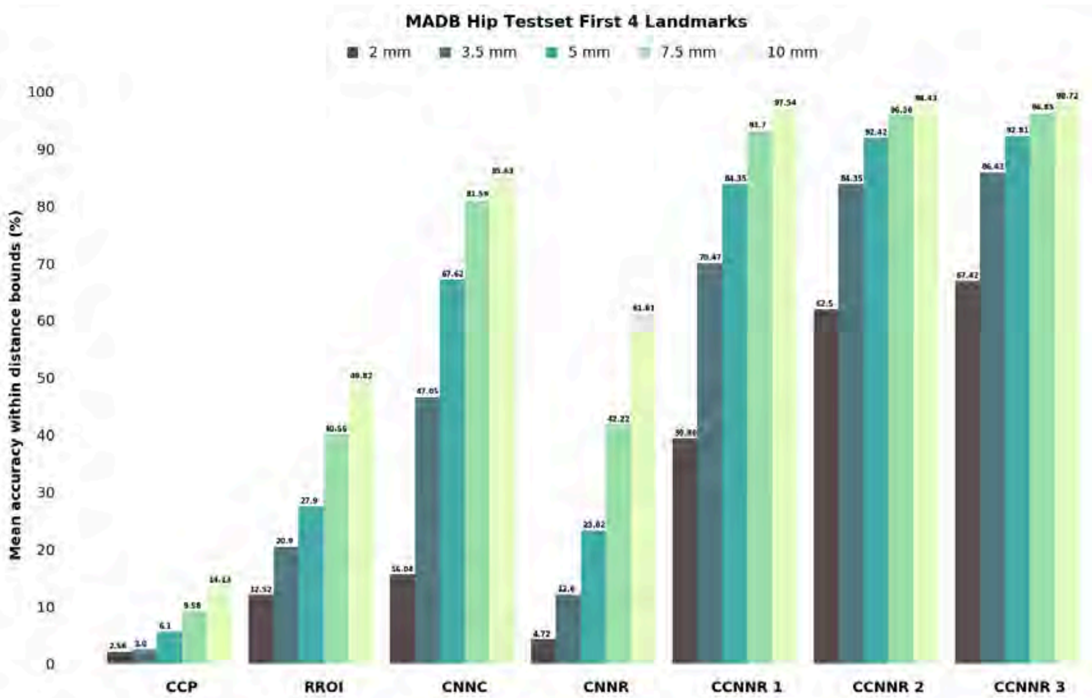

Figure 6.4: MADB on test set including the first 4 landmarks, reported on all methods

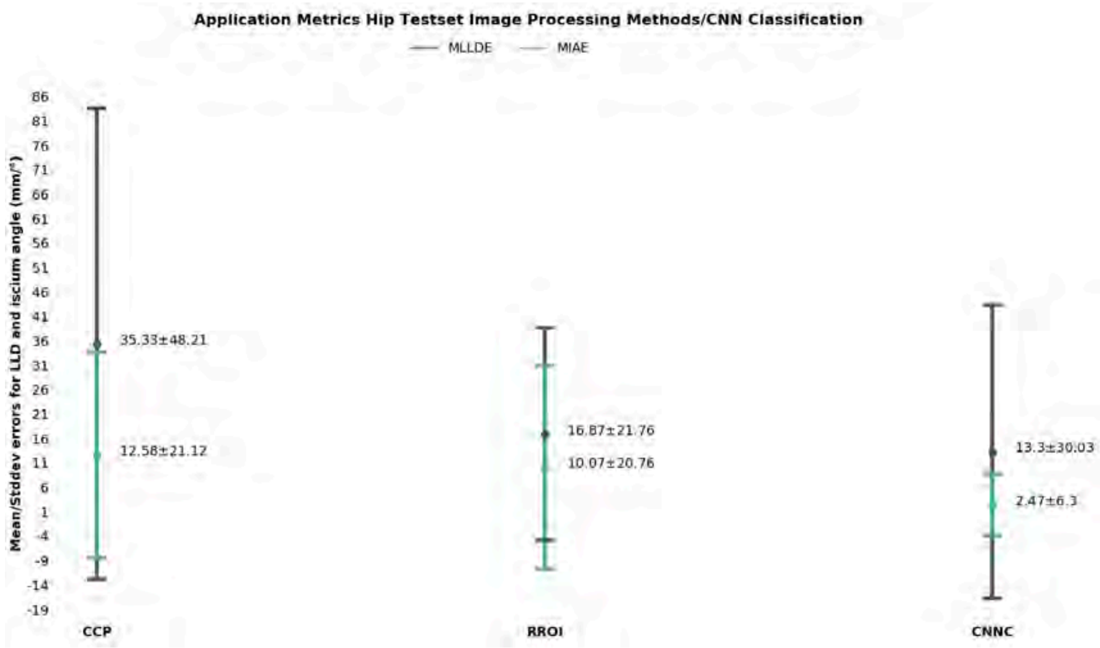

Figure 6.5: MLLDE and MIAE with standard deviations for image processing methods and CNNC

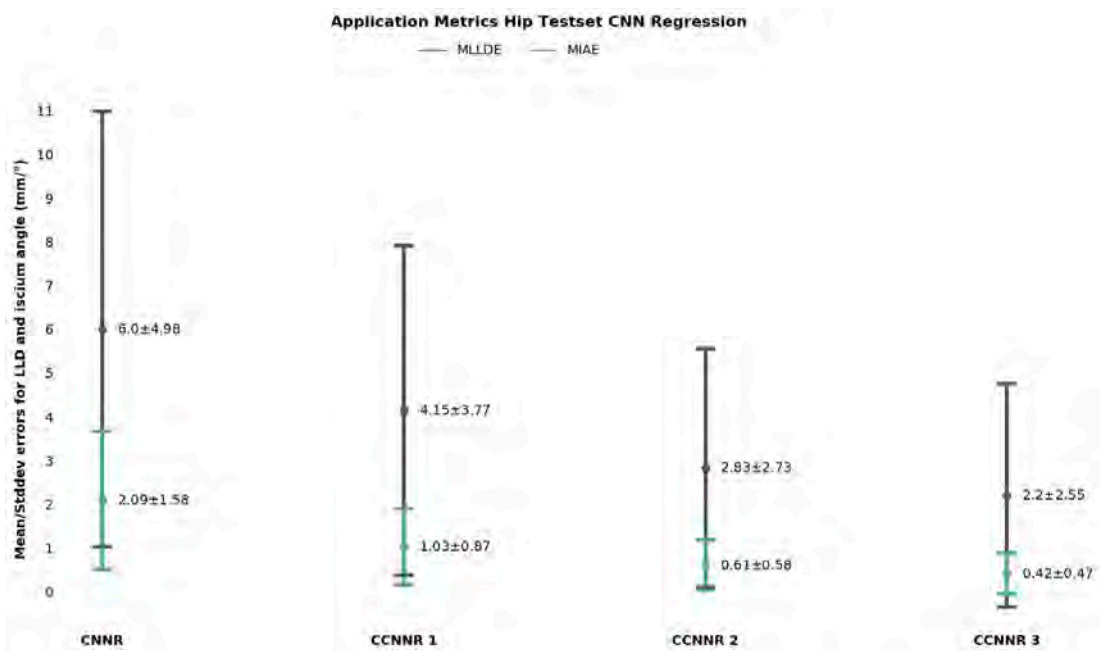

Figure 6.6: MLLDE and MIAE with standard deviations for CNN regressions

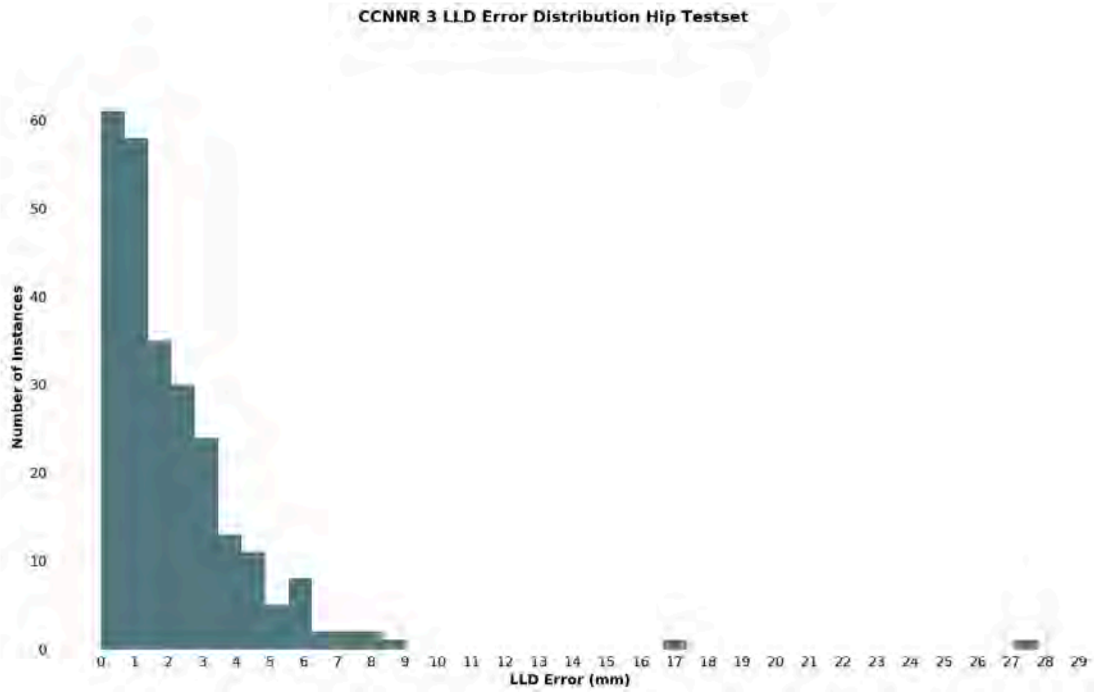

Figure 6.7: Histogram showing distribution of LLD error over the hip test set with CCNNR 3

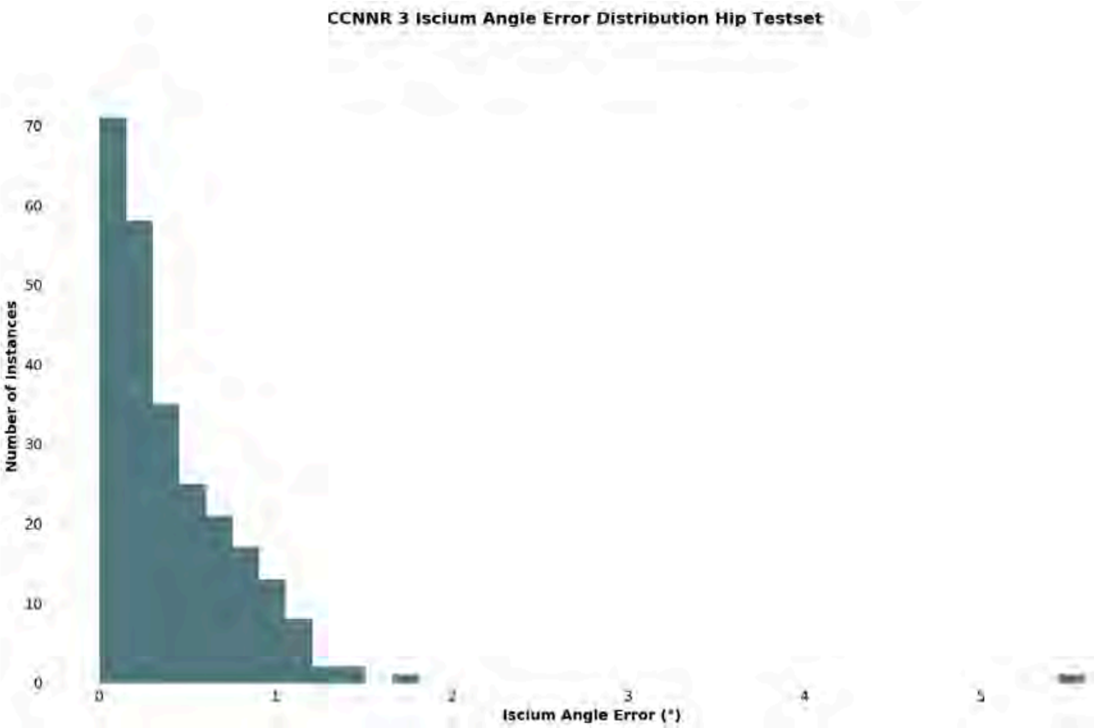

Figure 6.8: Histogram showing distribution of ischium angle error over the hip test set with CCNNR 3

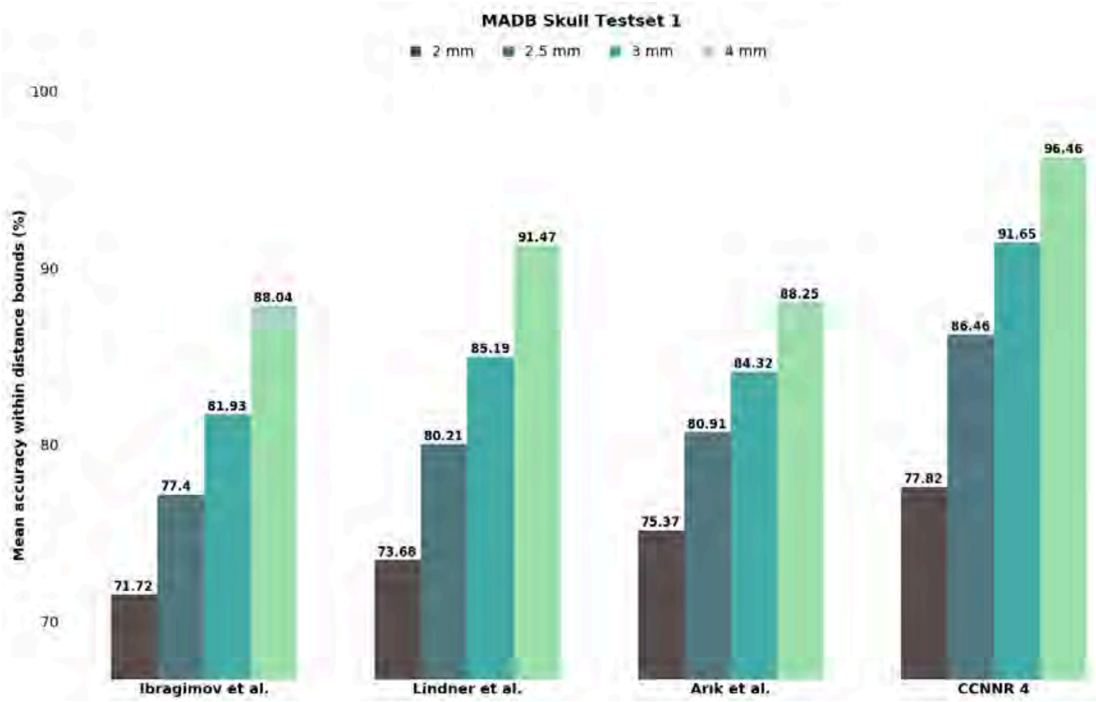

Figure 6.9: Comparison of CCNNR 4 result to other papers on IEEE ISBI 2015 Challenge test set 1

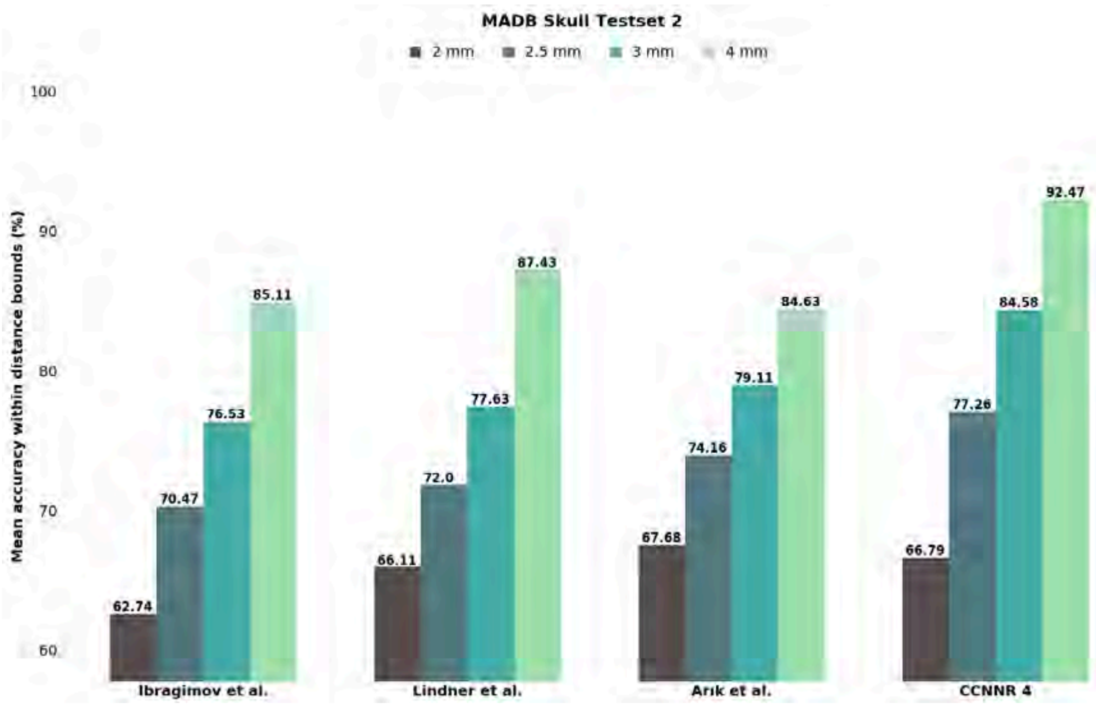

Figure 6.10: Comparison of CCNNR 4 result to other papers on IEEE ISBI 2015 Challenge test set 2

7 Discussion

In this chapter, methods will be compared based on previous evaluations. Hypotheses will be made about why some methods work better than others. In order to be well usable in production, not only algorithm performance is of importance. Therefore, implementation and UX aspects are discussed in a second subchapter.

7.1 Method Comparison

The CCP algorithm performed worse in all evaluations and finds only 2.56% of all landmarks in the 2 mm bound. Edges can often not be made visible. If canny thresholds are set too low, detailed but unnecessary contours become visible. On the other hand, setting it too high leaves major gaps far away from landmark coordinates. Low contrast landmark neighborhoods remain undetected. Furthermore, landmarks must not lie near change points which can be elucidated best on extreme cases like hidden trochanters, flat ischium or the caput center. CCP is also difficult to tweak. Training on many images generates an abundance of test instance features and thus candidates. In general, the number of candidates is not adjustable. It is not even guaranteed that one is returned if too few training images are utilized or the number of required neighbors yields an underrepresentation of features. The only advantage is easy intuition but in conclusion lacks practicability.

RROI performs better than CCP with 10.71% within 2 mm (6 landmarks). It always finds candidates even for landmarks 5/6 and is again easy to understand. Still, the algorithm has several drawbacks which are reflected in its results. The speed advantage from applying high strides is exchanged for high translation variance. This leads to inconsistent behavior, causes high standard deviations and makes hyperparametrization ungeneralizable. Another drawback is rotation sensitivity as the mean shape used for comparison represents mainly unrotated instances. Candidates can end up on the wrong side of the symmetric pelvis shape if the initial ROI size is too small (local view problem). The exact number of candidates cannot be controlled due to ROI redundancy in deeper layers. In order to mitigate this risk and to find distinct candidates, spawns should be > 1 only in early layers for wide spread. Finally, even though high strides are applied, sliding window approaches are still slow in prediction since no model is learned.

It is generally a good idea to apply binary classification with a CNN. The decision space is restricted to 2, many training instances can be generated out of a single image and CNNs are well known for their good results on classification tasks. CNNC achieves best theoretically possible accuracies with 75.8% within 2 mm (6 landmarks) even with a stride of 10px. Compared to final results, a huge fall is observed to 14.57%. The discrepancies shrink with higher bounds. It can be explained by considering that the stride forces candidates to have a minimum distance of $\sim$1.5 mm (dependent on pixel spacing) to each other. However, the shape model does not account for such changes appropriately and will still assign high likelihoods to (relatively) near candidates. One could then reason to increase the intensity likelihood factor. This on the other hand results in more side confusions due to pelvis symmetry (ischium, caput). The only possibilities to accomplish better results is hence to decrease stride to 1, resulting in excessive evaluation times as every pixel must be checked or to decrease image size. One advantage over previous discussed methods is the predictive power of CNNs. In addition, a certainty score is assigned to every pixel and must hence not be computed in retrospect when calculating payoffs. Finally, the number of candidates can be controlled.

Preceding methods rely on a hand-constructed complex shape model. It cannot account well for high variance cases like high LLDs. In addition, it adds time of polynomial complexity (see [35]) to select best candidate combinations. CNNR tackles the speed problem of CNNC as only one prediction must be conducted. Unfortunately, this effect diminishes again as we add a cascade and every landmark needs its own network. Including model loading and unoptimized code, a GPU needs on average 1.14s for CCNNR 3. A CPU without model loading requires 2.81s on average. A single image with CPU takes 3.52s. Timings for CCNNR 2 and 1 on a single image are 3.11 and 2.27s.

MADB after the initial regression is low with 5.25% within 2 mm (6 landmarks). It cannot compete with CNNC in any bound because images are too big and the dataset size does not suffice to account for it. Interestingly though, it preserves shape better with its implicit and time cost free shape model (compare MLLDE/MIAE) and is the principal argument for using cascades. After the first refinement, results improve considerably to 37.4% within 2 mm as many more training instances with new information can be sampled through shifting augmentations. Furthermore, patch sizes become smaller. Lower improvements can be seen along the cascade.

Comparing theoretically possible MADB values of CNNC with CCNNR 3, CNNC is better by 16.61 and 3.06% for bounds 2 and 3.5 mm but worse by 2.63, 5.15 and 5.83 % for bounds 5, 7.5 and 10 mm. This suggests that CNNC was not able to approach the correct landmark region in some occasions caused by single task learning. Estimating only first 4 landmarks tends to work slightly better than all 6 irrespective of method (e.g. 8.23% difference for CCNNR 3, 2 mm) except for CNNR. It could mean that landmarks 5 and 6 are harder to learn. It also leads to the hypothesis that CNNR performance increases the more landmarks are available for multi-task learning. Intuitively, knowing of landmark positions 5 and 6, predicting the first 4 landmarks becomes simpler.

MRE and MADB, particularly the lower bounds, should be viewed with caution. Those results are not always expressive especially for ischium and caput landmarks as they often do not possess a clear definition (flat ischium, not circle like caput). More expressive is the LLD and ischium angle error that set the first 4/2 landmarks in relation to each other. Best results are achieved with CCNNR 3 with a mean LLD error approaching the minimum mean of LLDs post surgery which varies from 1 to 15.9 mm in literature (see [40]) and a mean ischium angle error below 1°. The log distribution of application specific metrics reveals that most test instances are even below the already low means.

Comparing MREs between experts and CCNNR 3, the total mean of the machine is lower. On a landmark basis, experts achieved a slightly better mean value for landmarks 3 and 5. However, as the same experts have labeled the evaluated test data and the sample size of expert comparison is low, no meaningful conclusions can be drawn.

One disadvantage of cascades is that landmarks are not learned together after the initial regression which can lead to shape drifting. A fact that is rather unproblematic as long as landmarks are clearly defined from a local viewpoint. This is not the case for the caput center with no contrast given in small ROIs (imagine a zoomed version with landmark 5 as center). Looking at table 6.1 for landmark 5 confirms that hypothesis. From cascade 1 to 2, only minimal improvements can be observed. From cascade 2 to 3, MRE even increases slightly (and is higher than in CCNNR 1) in contrast to all other landmarks.

With results on skull data it is shown that this approach is also suitable for small datasets of only 150 images. In this case, the cascade is preferably deeper with a softer transition from complex to simple. The prediction jumps from first to second regression are farer but become again smaller (see typical example in appendix C) as more meaningful shifting augmentations can be performed. In row with this argumentation, the dataset size and image size is unproblematic as long as continuous improvement can be observed on ongoing smaller becoming ROIs, a trade-off between architecture depth and prediction time. The refinements/cascade can hence be interpreted as the system's ability to recover from its previous errors. In any case, a validation set with separate instances would further help to approximate the number of necessary ROIs, their sizes and maximal shifts for training with the presented heuristic.

Except for the 2 mm bound on test set 2, CCNNR 4 yield better results than previous approaches (as reported in [15]). The 90% mark has been reached on 3 mm in test set 1 as well as on testset 2 for 4 mm. As the clinically acceptable bound for cephalometry is merely 2 mm, it is argued [1] that automation is an unsolved problem. With optimization of the currently almost identical hip data architecture at present and/or more data provided, the current performance is expected to increase further and can definitely be used to semi-automate if not even fully automate cephalometry.

7.2 Product Integration

The goal is to make hip surgery planning easier, faster and more accurate. Placing landmarks is one of the few last manual steps in planning. The achieved results and computational speed of the CCNNR approach enable discussions about integration into Sectra's orthopedic package. In this section, some technicalities as well as user experience aspects with (dis-)/advantages are discussed.

Results should be reported and conceivably trained on data from different hospitals to draw conclusions about the models ability to generalize. Optimally, new images are automatically included for relearning existing models. With larger becoming datasets, techniques that allow for adjustment instead of complete relearning should be researched. Conceptually, new images must currently be scaled to the input size of the first CNN. Another option is to change the architecture to use techniques like global average pooling [41] to be scaling independent. Implementation wise, tensorflow/keras models and parameters need to be loaded/converted with tensorflow's C++ API which can then be integrated into C#. At present, only a python executable has been successfully called from C# code.

From a product point of view, several aspects should be addressed:

1. Automation grade
2. Automation control
3. Communication
4. Interface

Within the first point, the question above all is whether landmarks are still visible for users. To answer it, more data about how surgeons place landmarks and to which extent implants are moved after their position has been calculated based on those landmarks should be gathered. If it turns out that diverse adjustments are realized irrespective of placed landmarks, then it is more sensible to hide them. Implants are thus set per automatic labeling and surgeons adjust their sizes and positions. This implies that adjustments can be performed to every extent even with severe faulty detections. If users normally accept the implant placement calculated from landmarks, it is more sensible to let the option to manually alter landmarks suggestions.

Control over automation raises two questions. Will users be able to turn off automation? Do they need to activate it or is it automatically activated? If the feature is enabled manually under the user's consent, he must be informed about its existence in a proper way. An advantage of this path is that within the software, nothing changes for the user immediately. With automatic activation, users need to be educated about how landmarks/implants are adjusted appropriately. An option for turning off automation might be provided to let customers more control over what happens within the software.

Third, planning can have strong effects on the patient's wellbeing. After a couple of correct landmark suggestions, surgeon's trust in the system's automatisms might increase. They subsequently become more susceptible in accepting wrong suggestions. Neural networks are black boxes. Even though CNN's internal processes can be visualized, it is still difficult to explain them to non-experts. Any combination of jump distances (e.g. sum, product) within the cascade can give an indication about the model's certainty. Surgeons should still be informed that automatisms do not guarantee optimal planning. ALD should hence be communicated as solely giving a default from which to perform manual adjustments if necessary.

Lastly, manual labeling starts normally after entering hip planning. If landmark detection is activated by consent, a right click for opening the context menu to choose automatic labeling suffices and provides fast access. If manual landmark adjustments are possible, the user must be enabled to easily identify landmark ids as well as informed about the possibility to change their locations via affordances. One option is to select landmarks and drag (additional finer adjustments per keyboard direction keys) them to the corrected location. Graphics representing landmarks must in this case be big enough to allow selections. Another possibility is to register clicks of the environment near points.

8 Conclusion & Future Work

The goal of this thesis was to detect landmarks automatically on hip x-rays to support orthopedists in total hip replacement surgery planning. Therefore, 4 methods have been presented and were discussed in terms of accuracy, computational efficiency and production usage. Among these methods, CCP and RROI as image processing techniques exhibit high prediction errors and slow performance.

A binary classification convolutional neural network shows high theoretical accuracies but requires tremendous parallelization for the computation time demand within a few seconds in production. Unfortunately, the authors using CNNC [15] for skull landmark detection did not mention any times. Average times for previous methods reviewed in [1] range from 18 to 146 seconds for detecting 19 landmarks.

The investigated cascaded convolutional neural network regression approach together with the presented ROI size and training set composition heuristic proves oneself as fast and accurate. On average, 2,81s unoptimized on a CPU are needed to detect 6 landmarks. Even though efficiency is promising, the needed time increases along with cascade depth and number of landmarks. Solving tasks like landmark-based segmentation which might require hundreds of landmarks, results hence in higher computation times and task-specific as well as enhanced CNN techniques as e.g. in [13] might be considered. Consequent research should therefore be conducted in accelerating cascade based neural networks that simplify challenging problems continuously in a multi-task/non task-separating setting.

The cascaded approach achieved a mean leg length discrepancy error of 2.2 ± 2.55 mm and is thus below any LLD necessary to affect patients negatively [42]. With both time and evaluation results for CCNNRs, one can draw the conclusion that the above mentioned thesis goal has been fulfilled. However, only x-rays from people treated at the Linköping university hospital were used for training and evaluation. It would be interesting to research how well the model generalizes to x-rays from disparate sources and to expand the training set to possibly account for them. Some hospitals might use different x-ray machines/acquisition processes while people from other regions exhibit distinct pelvis variances. Furthermore, the inter-observer performance of surgeons should be assessed to allow stronger judgments about how the machine performs compared to them. Data acquisition might become easier in the future and models outperform humans in most cases. Surgeons will still predict rare cases much better when machines err on them. For that reason, more sophisticated data augmentation like elastic distortions or even generative adversarial networks might be utilized to expand data beyond what humans can naturally provide.

CCNNRs are applicable to all kinds of 2D automatic landmark detection and have been shown to work on big images and small dataset sizes as opposed to [23]. On the skull dataset (for automatic cephalometry), it outperforms previously best approaches in all but one distance bound by up to 6.46% without fine-tuning (adaption from hip data). Previous approaches [35, 43, 44, 45, 46] competing in the IEEE ISBI 2015 challenge rely in some form on hand-crafted features (e.g. Haar-like, Histogram of Oriented Gradient, Zernike). Even [15], in which visual appearance features are learned by a CNN, relies on a hand-crafted shape model. In the introductory text of chapter 3, landmark detection has been divided in the 3 tasks ROI extraction, landmark candidate selection based on visual features and final estimation by modeling of geometric relationships. While RROI, CCP and the described explicit shape model focus only on one of these steps, CCNNR unites them without hand-crafting features.

Comparing dataset sizes, the skull training set is 4 times smaller than the one for the hip and therefore utilized 4 refinements instead of 3. In general, it would be interesting to learn more about needed dataset sizes for specific problems in machine learning. For cascades, 4 research questions result:

1. Which function emerges when the training set size is assessed in relation to the number of cascades needed to achieve a desired performance?

2. Given a large training set, how would the prediction goodness differ comparing few vs. arbitrary many cascades?

3. Given a fixed number of cascades, how would results improve when adding more data to learn on?

4. Finally and most interestingly, would it be possible to learn on a single instance[1] and achieve good evaluation results?

Cascades simplify landmark detection as they are applied to smaller becoming regions. With deeper cascades, it is assumed that small datasets (e.g. skull data) can be tackled. Generalizing from these two hypotheses, it might be possible to solve landmark detection based on a single training image with the most extreme case of applying cascades with ROI sizes becoming smaller by 1 pixel. More broadly, techniques that simplify arbitrary regression as well as classification problems continuously for training data reduction would be well desired in the machine learning community (especially in the health care sector in which data acquisition is still a major problem). The collection of those techniques could be termed as deep simplification learning.

As next steps specifically for Sectra, the keras/tensorflow models need to be transferred to C# or C++ language for production integration. More test data from disparate sources should be used to prove wider applicability of trained networks. Training on more data leads to higher accuracies of the first regression and requires less refinements and hence a computational speedup. X-rays have been filtered to only include preop cases and could be expanded to postop. Better hardware and architecture refinements (gradient checkpointing [48] and synthetic gradients [49] for deeper CNNs, global average pooling [41], lime [50]/class activation maps [51] for visualization) can further improve results with a lower memory footprint, foster general applicability on different image sizes and provide better reasoning capabilities about a network's output. After functionality has been proven on hips, expansion to knee, shoulder and spine is appropriate. Research on the applicability for 3D planning can furthermore increase surgeon productivity.

[1] This is called one-shot learning, learn more in [47]

A Challenging X-Rays

Landmark detection on the pelvis is challenging because anatomy changes between patients. In some cases, it is even hard for humans to estimate these positions. In this appendix, several cases demonstrate various difficulties encountered in the dataset.

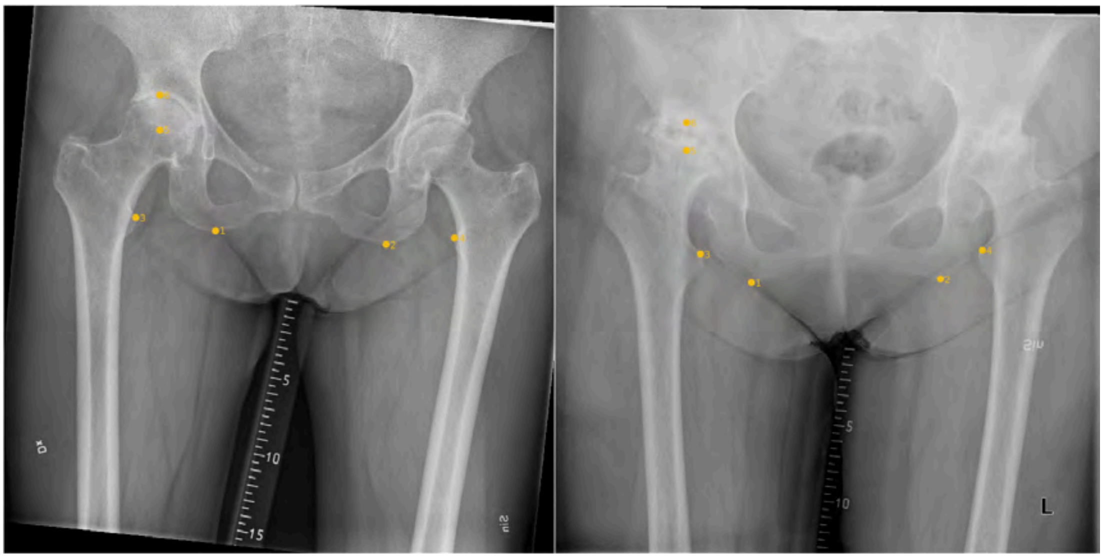

Figure A.1: Left: Rotations, hidden left trochanter minor, objects like rulers, right: weak contrast for landmarks 1 and 2, caput center and acetabular roof difficult to label

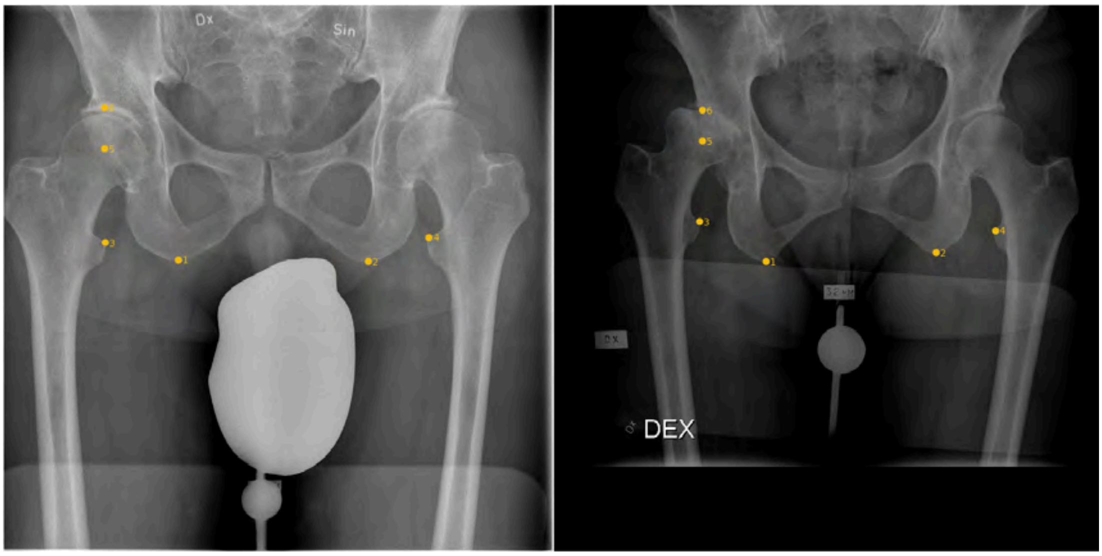

Figure A.2: Left: Protective gonad shield, right: Dark as well as very bright images

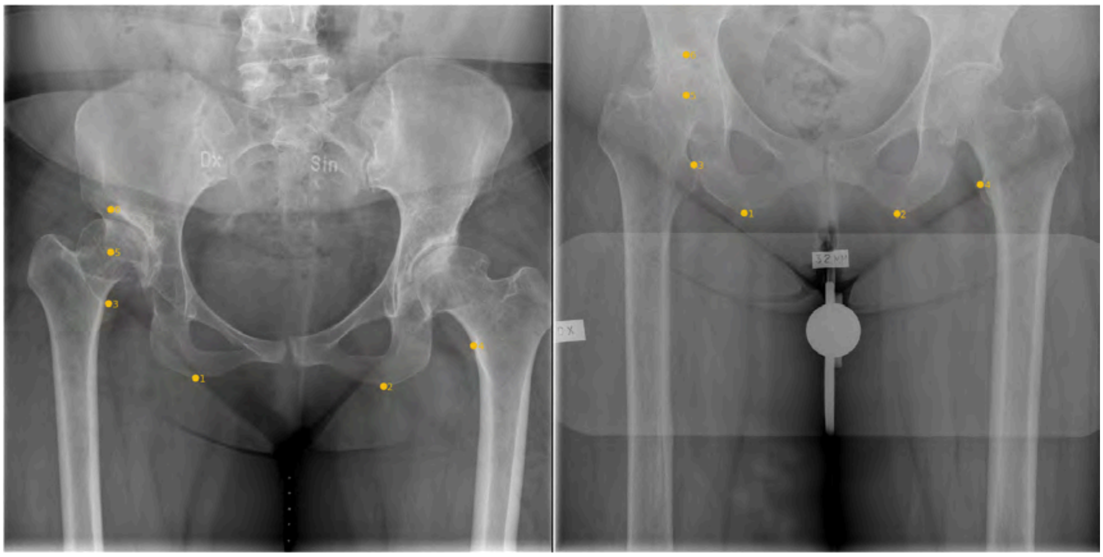

Figure A.3: Left: Translation showing the upper pelvis which is underrepresented in the dataset, right: Right trochanter minor and ischium rub against each other, no clear space between them as in most other cases.

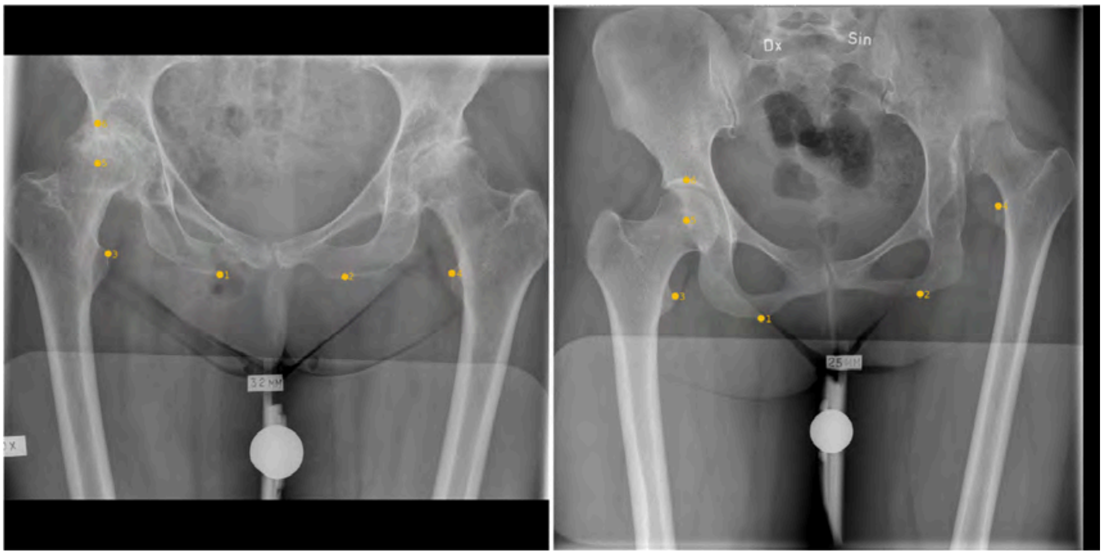

Figure A.4: Left: Top and bottom without visual information (black), flat ischium (no clear landmark definition), right: Rare case in which landmark 4 is above landmark 5 (no geometrical constraints and rules applicable)

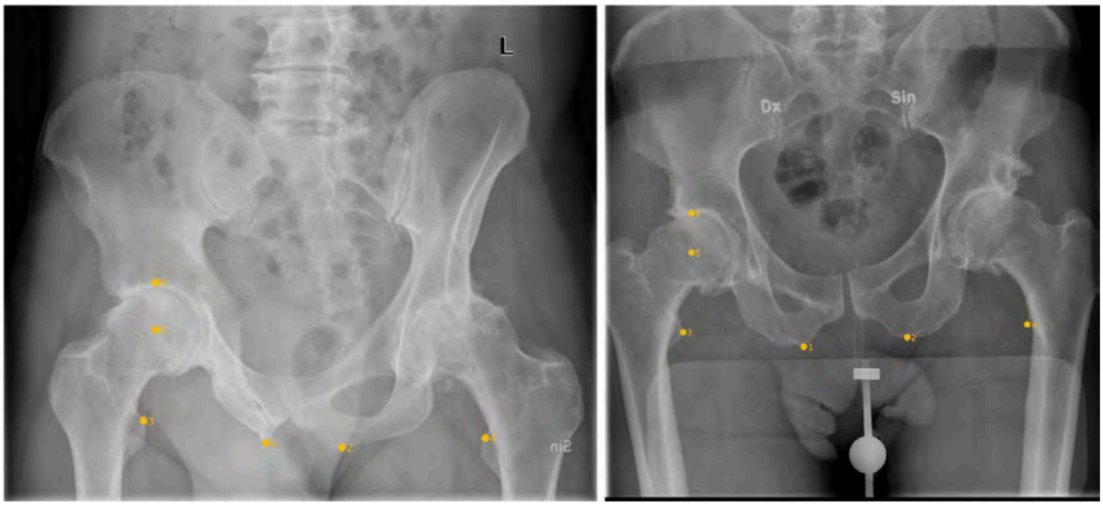

Figure A.5: Left: Pelvis symmetry lost, again a translation showing different anatomy compared to most other x-rays, right: Gender variances, also in this case, one trochanter minor is invisible.

B CCNNR Prediction Paths

All 254 test images including prediction paths as already presented in chapter 3.5 have been plotted to a pdf file. In this appendix, most interesting cases are selected including the few existing bad as well as good predictions on difficult cases (good and bad not based on metrics, just valued by viewing).

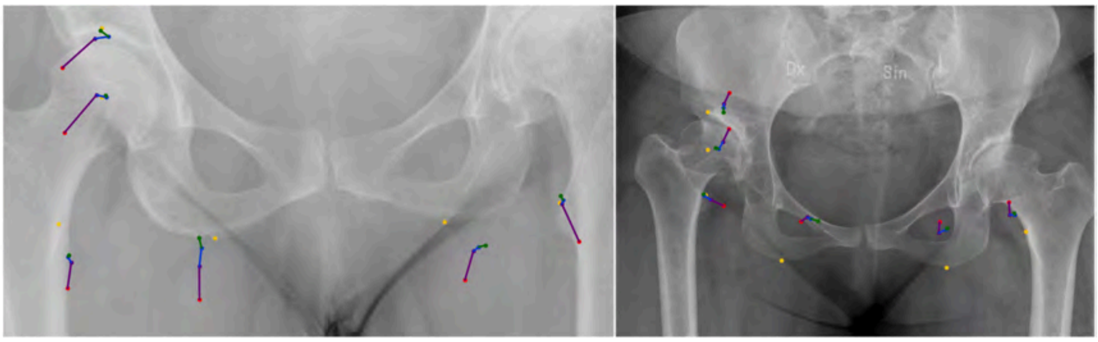

Figure B.1: Worst performing examples; left: Failed to spot hidden trochanter minor and ischium due to low contrast, right: Almost all predictions are far off because only few images in the dataset show the upper pelvis part.

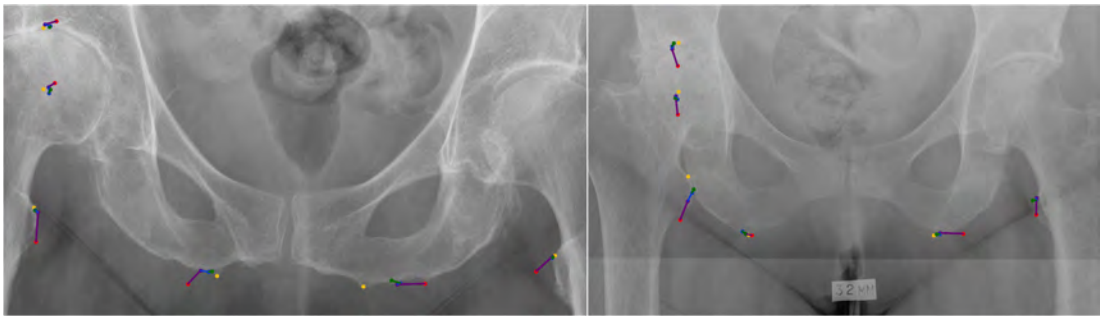

Figure B.2: Again rather bad predictions, left: Flat ischium causes wrong position, LLD not affected, right: Rubbing bones are rare cases. It is hence difficult to account for them.

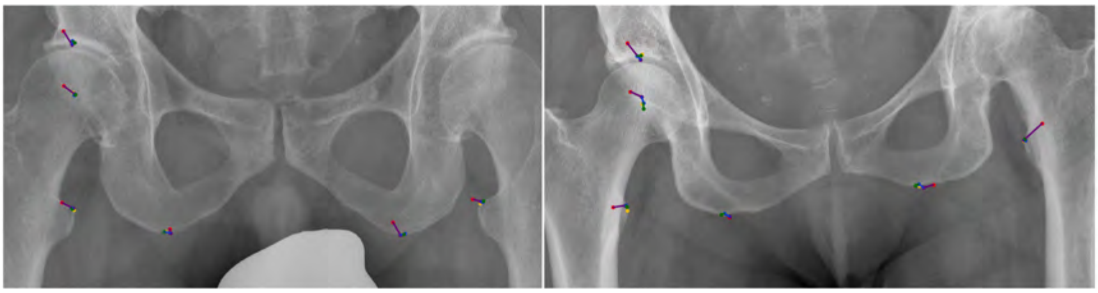

Figure B.3: Good examples, left: Gonad shield at the bottom does not interfere with predictions, right: Method accounts for rotations.

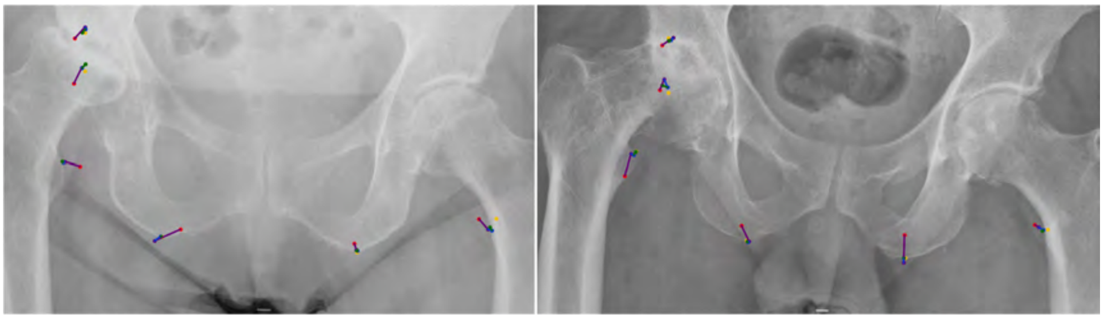

Figure B.4: Difficult examples, left: Hidden trochanter minor and caput without joint space, right: Rotated hidden trochanter detected.

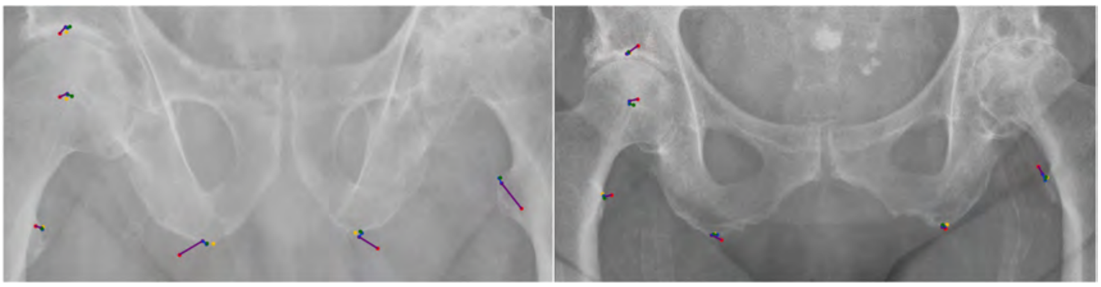

Figure B.5: Left: Acetabular roof landmark prediction better than ground truth, right: Another example of correct hidden trochanter estimation.

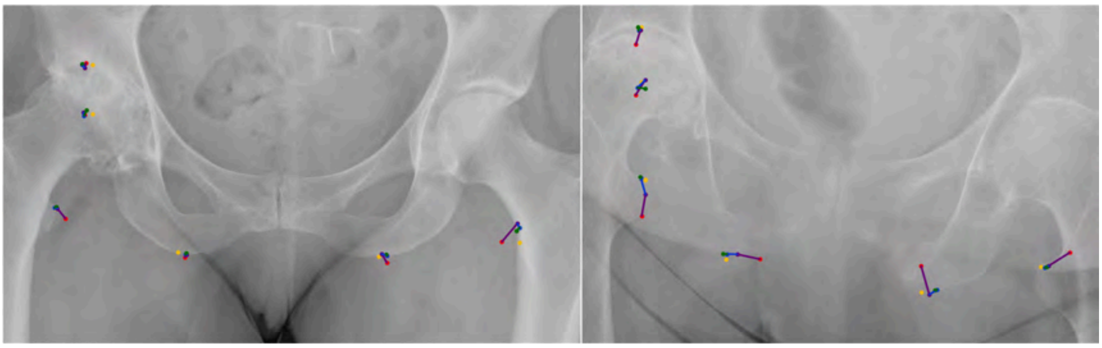

Figure B.6: Left: The hidden trochanter might be estimated more accurately than the true coordinate considering the symmetric shape of the pelvis, right: Remarkable performance on low contrast that is difficult to spot even for humans.

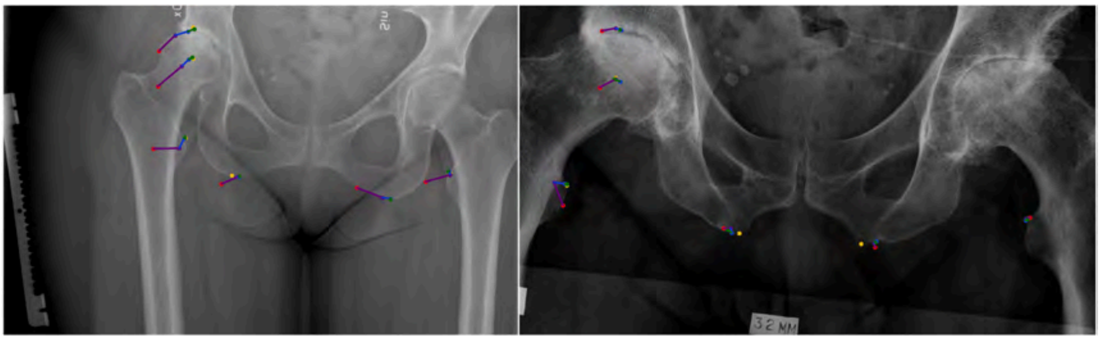

Figure B.7: Left: Comparably big refinement necessary due to uncommon object in the x-ray. right: Handling dark and bright images is unproblematic.

C Cephalometric Landmark Detection Details

The two test sets comprise 150 and 100 images. The image size for every image is 2400x1935 pixels. The pixel spacing is 0.1 mm. Networks have only been learned on training data (again 150 images) and 4 refinement regressions were applied. This is one regression more than used on hip data. The reason is that no validation data is accessible and the training set size is small.

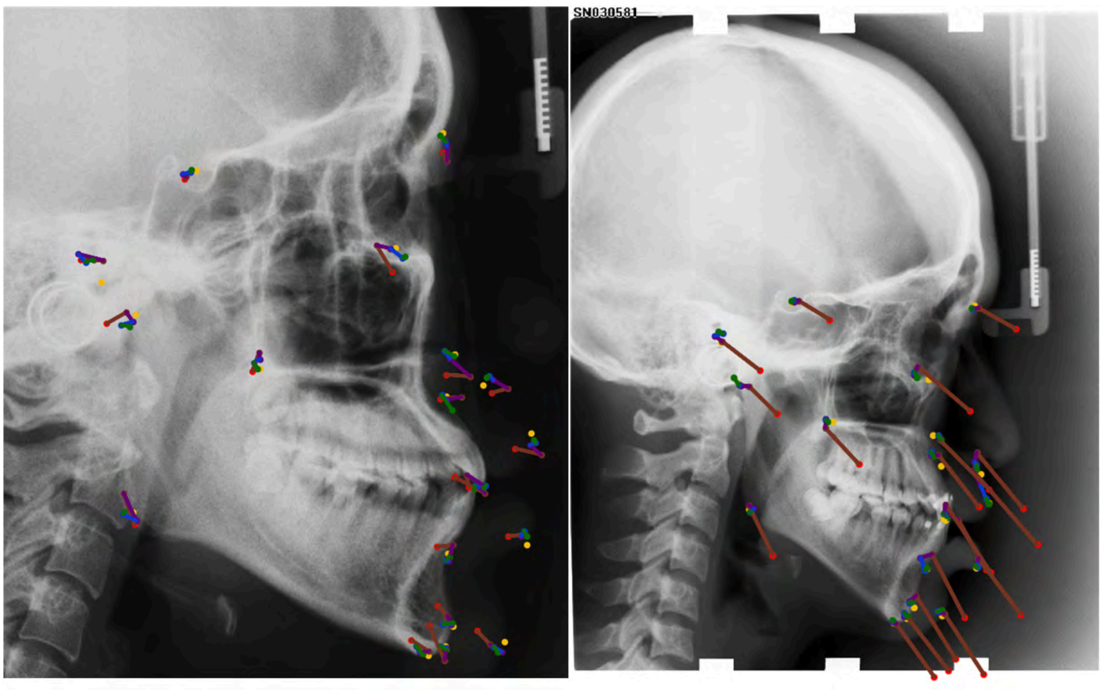

Figure C.1: Skull prediction paths; left: uncommon example with no distant jumps, right: common example with big jumps, 3 predictions even originate outside the image boundary

The first regression is, as previously, run on whole images. Generally, 50 epochs were used at the maximum based on best mean absolute error on training data. Augmentations are used as before resulting in 750 instances. For the first region, s_{ROI} is set to 1001 px ($d_h = 500$ px) with $shift_{max} = 300$ px. No real heuristic is applied as no validation data is available. For the upcoming ROIs, same hyper-parameters as for the hip are chosen. As validation data, more random samples are drawn from the actual training set. Training and validation data sizes for all refinements are (3000, 1500). Only for the first refinement, (1050, 300) has been chosen to save time and disk space.

CNN settings are almost equal to the hip setting, no fine-tuning to the particular dataset has been done. Since the initial skull images are bigger, strides are set to (4,2,2,1). The first refinement CNN applies the same settings as the first hip refinement CNN but has strides (4,2,1,1) and a batch size of 1.

Two example predictions are shown in image C.1. Often, big jumps from first regression to first refinement can be observed. Adding more (different, not only augmented) training instances will likely decrease them similar to hip data.

List of Abbreviations

CNN Convolutional Neural Network

CCP Contour Change Point

ROI Region of Interest

RROI Recursive Region of Interest

CNNC Convolutional Neural Network Classification

CNNR Convolutional Neural Network Regression

CCNNR Cascaded Convolutional Neural Network Regression

CC Change Code

CP Change Point

FDB Feature Database

conv convolutional

FC fully-connected

ReLU Rectified Linear Unit

THR Total Hip Replacement

LLD Leg Length Discrepancy

MRE Mean Radial Error

MSE Mean Squared Error

DO Dropout

ED Euclidean Distance

API Application programming interface

IEEE Institute of Electrical and Electronics Engineers

ISBI International Symposium on Biomedical Imaging

MADB Mean Accuracy within Distance Bound

MLLDE Mean Leg Length Discrepancy Error

MIAE Mean Iscium Angle Error

ALD Automatic Landmark Detection

LM Landmark

CT Computerized Tomography

MR Magnetic Resonance

PACS Picture Archiving and Communication System

DICOM Digital Imaging and Communications in Medicine

TIFF Tagged Image File Format

ML Machine Learning

RF Random Forest

GT Ground Truth

kNN k-Nearest Neighbor

SSD Sum of Squared Differences

CE Cross Entropy

KDE Kernel Density Estimation

MTL Multi-Task Learning

UX User Experience

CPU Central processing unit

GPU Graphics processing unit

List of Figures

Bibliography

[1] Wang, Ching-Wei, Cheng-Ta Huang, Meng-Che Hsieh, Chung-Hsing Li, Sheng-Wei Chang, Wei-Cheng Li, Rémy Vandaele, Raphaël Marée, Sébastien Jodogne, Pierre Geurts et al.: *Evaluation and comparison of anatomical landmark detection methods for cephalometric X-ray images: a grand challenge*. IEEE transactions on medical imaging, 34(9):1890–1900, 2015.

[2] OECD: *Health at a Glance 2017*.

[3] Jared R. H. Foran, MD: *Total Hip Replacement*. Web, 08 2015. https://orthoinfo.aaos.org/en/treatment/total-hip-replacement/. Accessed: 22.01.2018.

[4] Eggli, S, M Pisan and ME Müller: *The value of preoperative planning for total hip arthroplasty*. J Bone Joint Surg Br, 80(3):382–390, 1998.

[5] Chu, Chengwen, Cheng Chen, Li Liu and Guoyan Zheng: *Facts: fully automatic ct segmentation of a hip joint*. Annals of biomedical engineering, 43(5):1247–1259, 2015.

[6] Lindner, Claudia, S Thiagarajah, J Mark Wilkinson, Gillian A Wallis, Timothy F Cootes, arcOGEN Consortium et al.: *Accurate fully automatic femur segmentation in pelvic radiographs using regression voting*. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 353–360. Springer, 2012.

[7] Ehrhardt, Jan, Heinz Handels, Bernd Strathmann, Thomas Malina, Werner Plötz and Siegfried J Pöppl: *Atlas-based recognition of anatomical structures and landmarks to support the virtual three-dimensional*

planning of hip operations. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 17–24. Springer, 2003.

[8] Ehrhardt, J, H Handels, W Plötz, SJ Pöppl et al.: *Atlas-based recognition of anatomical structures and landmarks and the automatic computation of orthopedic parameters.* Methods Archive, 43(4):391–397, 2004.

[9] Wei, Guo-Qing, Jianzhong Qian and Helmuth Schramm: *System and method for the detection of anatomic landmarks for total hip replacement*, January 6 2004. US Patent 6,674,883.

[10] Betke, Margrit, Harrison Hong, Deborah Thomas, Chekema Prince and Jane P Ko: *Landmark detection in the chest and registration of lung surfaces with an application to nodule registration.* Medical Image Analysis, 7(3):265–281, 2003.

[11] Han, Dong, Yaozong Gao, Guorong Wu, Pew-Thian Yap and Dinggang Shen: *Robust anatomical landmark detection with application to MR brain image registration.* Computerized Medical Imaging and Graphics, 46:277–290, 2015.

[12] Zheng, Yefeng, Matthias John, Rui Liao, Jan Boese, Uwe Kirschstein, Bogdan Georgescu, S Kevin Zhou, Jörg Kempfert, Thomas Walther, Gernot Brockmann et al.: *Automatic aorta segmentation and valve landmark detection in C-arm CT: application to aortic valve implantation.* In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 476–483. Springer, 2010.

[13] Wu, Hongbo, Chris Bailey, Parham Rasoulinejad and Shuo Li: *Automatic Landmark Estimation for Adolescent Idiopathic Scoliosis Assessment Using BoostNet.* In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 127–135. Springer, 2017.

[14] Wang, Ching-Wei, Cheng-Ta Huang, Jia-Hong Lee, Chung-Hsing Li, Sheng-Wei Chang, Ming-Jhih Siao, Tat-Ming Lai, Bulat Ibragimov, Tomaž Vrtovec, Olaf Ronneberger et al.: *A benchmark for comparison of dental radiography analysis algorithms.* Medical image analysis, 31:63–76, 2016.

[15] Arik, Sercan Ö, Bulat Ibragimov and Lei Xing: *Fully automated quantitative cephalometry using convolutional neural networks.* Journal of Medical Imaging, 4(1):014501–014501, 2017.

[16] Ibragimov, Bulat, Boštjan Likar, Franjo Pernus et al.: *A game-theoretic framework for landmark-based image segmentation.* IEEE Transactions on Medical Imaging, 31(9):1761–1776, 2012.

[17] Litjens, Geert, Thijs Kooi, Babak Ehteshami Bejnordi, Arnaud Arindra Adiyoso Setio, Francesco Ciompi, Mohsen Ghafoorian, Jeroen AWM van der Laak, Bram van Ginneken and Clara I Sánchez: *A survey on deep learning in medical image analysis.* arXiv preprint arXiv:1702.05747, 2017.

[18] Zhu, Xiangxin and Deva Ramanan: *Face detection, pose estimation, and landmark localization in the wild.* In *Computer Vision and Pattern Recognition (CVPR), 2012 IEEE Conference on*, pages 2879–2886. IEEE, 2012.

[19] Pantic, Maja and Leon J. M. Rothkrantz: *Automatic analysis of facial expressions: The state of the art.* IEEE Transactions on pattern analysis and machine intelligence, 22(12):1424–1445, 2000.

[20] Wang, Patricia, Xiaofeng Tong, Yangzhou Du, Jianguo Li, Wei Hu and Yimin Zhang: *Augmented makeover based on 3D morphable model.* In *Proceedings of the 19th ACM international conference on Multimedia*, pages 1569–1572. ACM, 2011.

[21] Azevedo, Pedro, Thiago Oliveira Dos Santos and Edilson De Aguiar: *An Augmented Reality Virtual Glasses Try-On System.* In *Virtual and Augmented Reality (SVR), 2016 XVIII Symposium on*, pages 1–9. IEEE, 2016.

[22] Ramanathan, Narayanan and Rama Chellappa: *Modeling age progression in young faces.* In *Computer Vision and Pattern Recognition, 2006 IEEE Computer Society Conference on*, volume 1, pages 387–394. IEEE, 2006.

[23] Sun, Yi, Xiaogang Wang and Xiaoou Tang: *Deep convolutional network cascade for facial point detection.* In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3476–3483, 2013.

[24] Zhang, Zhanpeng, Ping Luo, Chen Change Loy and Xiaoou Tang: *Facial landmark detection by deep multi-task learning.* In *European Conference on Computer Vision*, pages 94–108. Springer, 2014.

[25] Zhang, Cha and Zhengyou Zhang: *Improving multiview face detection with multi-task deep convolutional neural networks.* In *Applications of Computer Vision (WACV), 2014 IEEE Winter Conference on*, pages 1036–1041. IEEE, 2014.

[26] Ranjan, Rajeev, Vishal M Patel and Rama Chellappa: *Hyperface: A deep multi-task learning framework for face detection, landmark localization, pose estimation, and gender recognition.* arXiv preprint arXiv:1603.01249, 2016.

[27] Yu, Xiang, Feng Zhou and Manmohan Chandraker: *Deep deformation network for object landmark localization.* In *European Conference on Computer Vision*, pages 52–70. Springer, 2016.

[28] Chen, Yu, Chunhua Shen, Xiu-Shen Wei, Lingqiao Liu and Jian Yang: *Adversarial Learning of Structure-Aware Fully Convolutional Networks for Landmark Localization.* arXiv preprint arXiv:1711.00253, 2017.

[29] Canny, John: *A computational approach to edge detection.* IEEE Transactions on pattern analysis and machine intelligence, (6):679–698, 1986.

[30] Gonzales, Rafael C. and Paul Wintz: *Digital Image Processing (2Nd Ed.).* Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1987.

[31] Trucco, Emanuele and Alessandro Verri: *Introductory techniques for 3-D computer vision*, volume 201. Prentice Hall Englewood Cliffs, 1998.

[32] Esteva, Andre, Brett Kuprel, Roberto A Novoa, Justin Ko, Susan M Swetter, Helen M Blau and Sebastian Thrun: *Dermatologist-level classification of skin cancer with deep neural networks.* Nature, 542(7639):115–118, 2017.

[33] Nielsen, Michael A: *Neural networks and deep learning*, 2015. `http://neuralnetworksanddeeplearning.com`.

[34] Goodfellow, Ian, Yoshua Bengio and Aaron Courville: *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[35] Ibragimov, Bulat, Boštjan Likar, Franjo Pernuš and Tomaž Vrtovec: *Automatic cephalometric X-ray landmark detection by applying game theory and random forests*. In *Proceedings ISBI*, pages 1–8, 2014.

[36] Caruana, Rich: *Multitask learning*. In *Learning to learn*, pages 95–133. Springer, 1998.

[37] Zhang, TY and Ching Y. Suen: *A fast parallel algorithm for thinning digital patterns*. Communications of the ACM, 27(3):236–239, 1984.

[38] Suzuki, Satoshi et al.: *Topological structural analysis of digitized binary images by border following*. Computer vision, graphics, and image processing, 30(1):32–46, 1985.

[39] Kingma, Diederik P. and Jimmy Ba: *Adam: A Method for Stochastic Optimization*. CoRR, abs/1412.6980, 2014.

[40] Konyves, A and GC Bannister: *The importance of leg length discrepancy after total hip arthroplasty*. Bone & Joint Journal, 87(2):155–157, 2005.

[41] Lin, Min, Qiang Chen and Shuicheng Yan: *Network in network*. arXiv preprint arXiv:1312.4400, 2013.

[42] Gurney, Burke: *Leg length discrepancy*. Gait & posture, 15(2):195–206, 2002.

[43] Chu, Chengwen, Cheng Chen, LP Nolte and G Zheng: *Fully automatic cephalometric x-ray landmark detection using random forest regression and sparse shape composition*. submitted to Automatic Cephalometric X-ray Landmark Detection Challenge, 2014.

[44] Chen, C and G Zheng: *Fully automatic landmark detection in cephalometric x-ray images by data-driven image displacement estimation*. In *Proc. ISBI Int. Symp. Biomed. Imag. 2014, Automat. Cephalometric X-Ray Landmark Detection Challenge*, pages 17–24, 2014.

[45] Mirzaalian, Hengameh and Ghassan Hamarneh: *Automatic globally-optimal pictorial structures with random decision forest based likelihoods for cephalometric x-ray landmark detection.* In *Proc. ISBI Int. Symp. Biomed. Imag. 2014, Automat. Cephalometric X-Ray Landmark Detection Challenge*, pages 25–36. Citeseer, 2014.

[46] Vandaele, Rémy, Raphaël Marée, Sébastien JODOGNE and Pierre Geurts: *Automatic cephalometric x-ray landmark detection challenge 2014: A tree-based algorithm.* ISBI, 2014.

[47] Fei-Fei, Li, Rob Fergus and Pietro Perona: *One-shot learning of object categories.* IEEE transactions on pattern analysis and machine intelligence, 28(4):594–611, 2006.

[48] Chen, Tianqi, Bing Xu, Chiyuan Zhang and Carlos Guestrin: *Training Deep Nets with Sublinear Memory Cost.* CoRR, abs/1604.06174, 2016.

[49] Czarnecki, Wojciech Marian, Grzegorz Swirszcz, Max Jaderberg, Simon Osindero, Oriol Vinyals and Koray Kavukcuoglu: *Understanding Synthetic Gradients and Decoupled Neural Interfaces.* CoRR, abs/1703.00522, 2017.

[50] Ribeiro, Marco Tulio, Sameer Singh and Carlos Guestrin: *"Why Should I Trust You?": Explaining the Predictions of Any Classifier.* In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, pages 1135–1144, 2016.

[51] Zhou, Bolei, Aditya Khosla, Agata Lapedriza, Aude Oliva and Antonio Torralba: *Learning deep features for discriminative localization.* In *Computer Vision and Pattern Recognition (CVPR), 2016 IEEE Conference on*, pages 2921–2929. IEEE, 2016.